The adjoining table on page 957 shows the BCD code. Each of the decimal digits (0 through 9) is represented by its binary equivalent. Since a decimal digit can be as large as 9, four bits are required to code each decimal digit (the binary code for 9 is 1001).

Note that each decimal digit is assigned a 4-bit binary number even though the binary equivalent may require fewer than four binary places. This way, circuits which use BCD code always handle the string of binary bits in four - place groups. When using BCD code, remember that all zeros must be retained, unlike a binary number where leading zeros can be dropped. The BCD code is used when it is necessary to transfer decimal information into and out of a digital machine. Examples of digital machines include the digital clocks, calculators, digital voltmeters and frequency counters.

Example 24.23. *What decimal number is represented by the BCD string given below ?*

0100 0000 0010

Solution. Divide the BCD number into 4-bit groups and convert each to decimal.

$$\underbrace{0100}_{4} \quad \underbrace{0000}_{0} \quad \underbrace{0010}_{2}$$

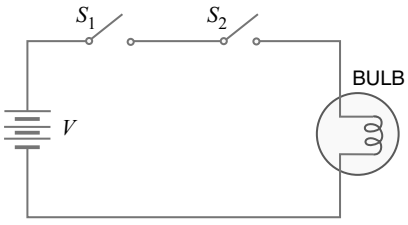
Therefore, the equivalent decimal number is $(402)_{10}$.

Note. To avoid confusion between BCD and true binary, a BCD string is often separated into groups of 4 binary bits or a subscript BCD is sometimes attached to the string as illustrated under :

0100 0000 0010 or 010000000010_{BCD}

24.17 Logic Gate

A **logic gate** *is a digital circuit that produces one output voltage which is either high or low, depending upon the levels (high or low) of input voltage/voltages.*



(*i*)

S_1	S_2	BULB
OPEN	OPEN	OFF
OPEN	CLOSED	OFF
CLOSED	OPEN	OFF
CLOSED	CLOSED	ON

(*ii*)

Truth Table

S_1	S_2	OUTPUT
0	0	0
0	1	0
1	0	0
1	1	1

(*iii*)

Fig. 24.10

A logic gate may have one or more inputs but only one output. Since a logic gate is a switching circuit (*i.e.,* a digital circuit), its output can have only one of the two possible states *viz.,* either a high voltage (1) or a low voltage (0) – it is either *ON* or *OFF*. Whether the output voltage of a logic gate is high (1) or low (0) will depend upon the conditions at its input. Fig. 24.10 (*i*) shows the basic idea of a *logic gate using switches.

(*i*) When S_1 and S_2 are open, the bulb is *OFF*.

* In itself, the circuit is not actually a logic gate but the logic is similar. The actual gate circuits are made with diodes and transistors. In other words, switches S_1 and S_2 are replaced by diodes or transistors.

(*ii*) When S_1 is open and S_2 closed, the bulb is *OFF*.

(*iii*) When S_2 is open and S_1 closed, the bulb is *OFF*.

(*iv*) When both S_1 and S_2 are closed, the bulb is *ON*.

Note that output (*OFF* or *ON*) depends upon the conditions at the input.

The four possible combinations of switches S_1 and S_2 are shown in the table in Fig. 24.10 (*ii*). It is clear that when either of the switches (S_1 or S_2) or both are open, the bulb is *OFF*. In binary language, when either of the inputs or both the inputs are low (0), the output is low. When both switches are closed, the bulb is *ON*. In terms of binary language, when both the inputs are high (1), the output is high. It is usual practice to show the conditions at the input and output of a logic gate in the binary form as shown in the table in Fig. 24.10 (*iii*). Such a table is called **truth table**.

The term "logic" is usually used to refer to a decision-making process. A logic gate makes logical decisions regarding the existence of output depending upon the nature of the input. Hence, such circuits are called logic circuits. The following points may be noted about the logic *gates :

(*i*) A binary 0 represents 0 V and binary 1 represents + 5 V**. It is common to refer to binary 0 as LOW input or output and binary 1 as HIGH input or output.

(*ii*) A logic gate has only one output signal. The output will depend upon the input signal/signals and the type of gate.

(*iii*) The operation of a logic gate may be described either by *truth table* or *Boolean algebra*.

24.18 Three Basic Logic Gates

The three basic logic gates that make up all the digital circuits are :

(*i*) OR gate

(*ii*) AND gate

(*iii*) NOT gate

In logic gates, we use junction diodes and transistors to perform switching functions. Each logic gate is represented by a graphic symbol (called logic symbol) and its function is defined by truth table or by a Boolean expression.

24.19 OR Gate

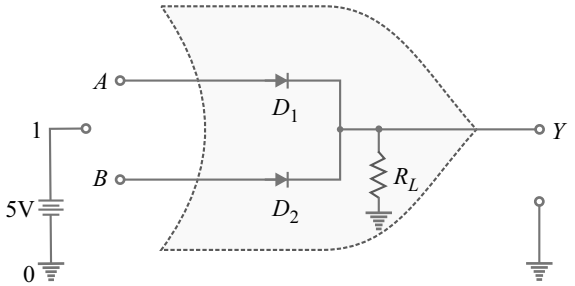
An **OR gate** *is a logic gate that has two or more inputs but only one output. It gives high output (1) if any or all the inputs are high (1) otherwise the output is low (0).*

It is called OR gate because the output is HIGH if any or all the inputs are HIGH. For the same reason, an OR gate is sometimes called "any or all gate". For example, consider a 2-input OR gate. The output *Y* will be HIGH if either or both inputs are HIGH.

---

* A *gate* can be regarded as a *barrier* which when closed prevents the passage of information but if open allows the signal/signals to pass through freely.

** In digital systems, the binary information is represented by two voltage levels, generally + 5 V and 0 V. So 5 V is used to represent binary 1 and 0 V is used to represent binary 0.



(*i*)

Truth table

A	*B*	*Y*
0	0	0
0	1	1
1	0	1
1	1	1

(*ii*)

Fig. 24.11

OR gate operation. Fig. 24.11 (*i*) shows two-input OR gate built by using two ideal diodes D_1 and D_2 and a resistor R_L. The input voltages are labelled as A and B while the output voltage is Y. Note that negative terminal of the battery is grounded and corresponds to 0 state (LOW level). The positive terminal of the battery (+ 5 V) corresponds to 1 state (HIGH level). There are only four input-output possibilities.

(*i*) When $A = 0$ and $B = 0$. This means that both diodes D_1 and D_2 are connected to ground (0V). Therefore, both the diodes are reverse biased and non-conducting. As a result, output voltage across R_L is zero *i.e.,* $Y = 0$ as shown in the truth table in Fig. 24.11 (*ii*).

(*ii*) When $A = 0$ and $B = 1$. This means that D_1 is connected to ground (0 V) and D_2 is connected to + 5 V. Therefore, D_1 does not conduct (reverse biased) while D_2 conducts (forward biased). As a result, full voltage of + 5 V appears across the load R_L. In other words, output is high *i.e.,* $Y = 1$ as shown in the truth table in Fig. 24.11 (*ii*).

(*iii*) When $A = 1$ and $B = 0$. This means that diode D_1 is connected to + 5 V and diode D_2 to ground (0 V). Therefore, D_1 is forward biased and conducts while D_2 is reverse biased and does not conduct. As a result, voltage across R_L is again + 5 V. In other words, output is high *i.e.,* $Y = 1$ as shown in the truth table in Fig. 24.11 (*ii*).

(*iv*) When $A = 1$ and $B = 1$. This means that both diodes D_1 and D_2 are connected to + 5 V. Therefore, both diodes are forward biased and conduct. Since the diodes are in parallel, the output voltage is + 5 V. In other words, the output is high *i.e.,* $Y = 1$ as shown in the truth table in Fig. 24.11 (*ii*).

It is clear from the truth table that *for OR gate, the output is high if any or all of the inputs are high. The only way to get a low output is by having all inputs low.* Fig. 24.12 shows the logic symbol of OR gate. Note that the symbol has curved line at the input.

Boolean expression. The algebra used to symbolically describe logic functions is called Boolean algebra. The "+" sign in Boolean algebra refers to the logical OR function. The Boolean expression for OR function is

$$A + B = Y$$

↑

OR symbol

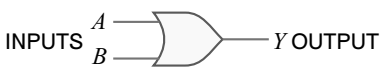


Fig. 24.12

$A + B$	=	Y
0 + 0	=	0
0 + 1	=	1
1 + 0	=	1
1 + 1	=	1

This expression is read as *Y equals A or B* **not** *Y equals A plus B.*

The adjoining truth table to Fig. 24.12 shows how OR gate functions for the two inputs. It is clear from the truth table that when 0 is ORed with 0, the result equals 0. Also, any variable ORed with 1 equals 1. The OR function can be summed up as under :

0 ORed with 0 equals 0

0 ORed with 1 equals 1

1 ORed with 1 equals 1

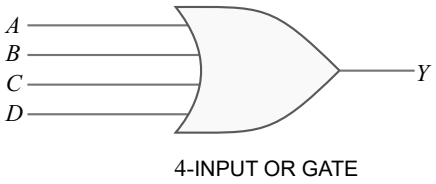


Fig. 24.13

An OR gate may have more than two inputs but there is only one output. Thus Fig. 24.13 shows four-input OR gate. The boolean expression for this four-input OR gate is

$$A + B + C + D = Y$$

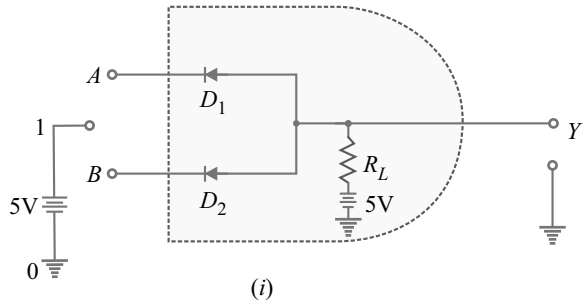
Note. The number of possible input combinations (rows) in a truth table is equal to 2^n where n is the number of input variables. For example, an eight-input gate would require $2^8 = 256$ rows for its truth table.

24.20 AND Gate

An **AND gate** *is a logic gate that has two or more inputs but only one output, it gives high output (1) when all the inputs are high (1), otherwise the output is low (0).*

It is called AND gate because output is HIGH only when all the inputs are HIGH. For this reason, the AND gate is sometimes called "all or nothing gate". For example, consider a 2-input AND gate. The output will be HIGH when both the inputs are HIGH.

AND gate operation. Fig. 24.14 (*i*) shows two-input AND gate built by using two ideal diodes D_1 and D_2 and a resistor R_L. There are four input-output possibilities.



(*i*)

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

(*ii*)

Fig. 24.14

(*i*) When *A* = 0 and *B* = 0. This means that both diodes D_1 and D_2 are connected to the ground (0 V). Both the diodes are forward biased and conduct current. As a result, the two diodes are shorted so that Y also gets grounded through these shorted diodes. Hence output is zero *i.e.,* $Y = 0$ as shown in the truth table in Fig. 24.14. (*ii*).

(*ii*) When $A = 0$ and $B = 1$. This means that diode D_1 is connected to ground (0 V) and diode D_2 is connected to + 5 V. Therefore, D_1 conducts (forward biased) and is grounded while D_2 does not conduct. Hence output is zero *i.e.,* $Y = 0$ as shown in the truth table in Fig. 24.14 (*ii*).

(*iii*) When $A = 1$ and $B = 0$. This means that diode D_1 is connected to + 5 V and diode D_2 is connected to ground (0 V). Therefore, the roles of diodes are interchanged as compared to the case (*ii*). Hence output is again zero *i.e.,* $Y = 0$ as shown in the truth table in Fig. 24.14 (*ii*).

(*iv*) When $A = 1$ and $B = 1$. This means that both diodes D_1 and D_2 are at + 5 V and do not conduct any current. Now output voltage is + 5 V because there is no current through R_L. Hence output is high *i.e.,* $Y = 1$ as shown in the truth table in Fig. 24.14 (*ii*).

It is clear from the truth table that *for AND gate, the output is high if all the inputs are high. However, the output is low if any or all the inputs are low.*

Fig. 24.15 shows the logic symbol of AND gate having two inputs A and B and output Y. The Boolean expression for two-input AND gate shown in Fig. 24.15 is

$$A \cdot B = Y$$

↑

AND Symbol



Fig. 24.15

The dot between A and B is the symbol for AND operation. This expression is read as "Y equals A and B". Quite often the dot is omitted and the expression is written as $AB = Y$. However, the expression $AB = Y$ is still read as Y equals A and B.

$A \cdot B = Y$
$0 \cdot 0 = 0$
$0 \cdot 1 = 0$
$1 \cdot 0 = 0$
$1 \cdot 1 = 1$

The adjoining truth table shows how AND gate functions for two inputs. It is clear that 0 ANDed with any variable equals 0. Also 1 ANDed with 1 equals 1. The AND function can be summed up as under :

0 ANDed with 0 equals 0

0 ANDed with 1 equals 0

1 ANDed with 1 equals 1

An AND gate may have more than two inputs but there is only one output. Thus Fig. 24.16 shows three-input AND gate. The boolean expression for this three-input AND gate is

$$A \cdot B \cdot C = Y$$



Fig. 24.16

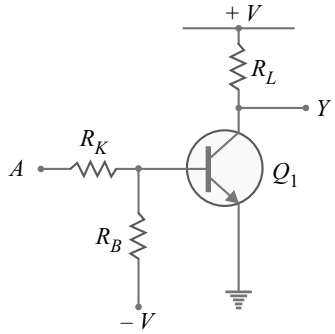
Note that the output Y of AND gate will be high (1) when all of the inputs are high (1) otherwise the output will be low.

24.21 NOT Gate (Or Inverter)

A **NOT gate** *is a logic gate with one input and one output. It gives high output (1) if the input is low (0) and vice-versa.*

NOT gate operation. The NOT gate is often called *inverter* because it inverts the input *i.e.,* output is opposite of the input. Fig. 24.17 (*i*) shows the inverter or NOT circuit. Here R_K and R_B form a voltage divider between ground and the negative voltage ($-V$). The operation of NOT gate circuit is as under :

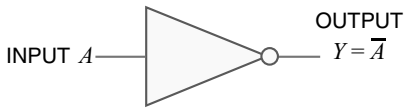
(*i*) When input $A = 0$. When A (the base of the transistor) is connected to ground, the base of the transistor Q_1 will become negative. This makes base-emitter junction reverse biased so that base current and hence collector current is zero. Therefore, output voltage is $+V$ volts. Hence the output is high *i.e.,* $Y = 1$ as shown in the truth table in Fig. 24.17 (*ii*).



(*i*)

A	*Y*
0	1
1	0

(*ii*)



(*iii*)

Fig. 24.17

(*ii*) When input $A = 1$. If sufficiently large positive voltage is applied at A, the base-emitter junction and collector-base junction become forward biased and the transistor comes in the *saturated mode*. Under this condition, maximum collector current flows. Therefore, output voltage becomes zero. Hence the output is low *i.e.,* $Y = 0$ as shown in the truth table in Fig. 24.17 (*ii*). It is clear from the truth table that *for a NOT gate, the output is high (1) if input is low (0) and vice-versa*.

Fig. 24.17 (*iii*) shows the logic symbol for NOT gate or inverter. Note that small bubble on the inverter symbol represents inversion. The Boolean expression for NOT function is

$$Y = \overline{A}$$

Note that bar above the input A represents inversion.

$$\text{If } A = 0, \text{ then } Y = \overline{0} \text{ or } Y = 1$$

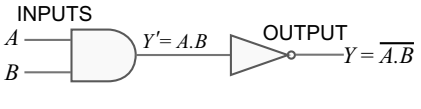
$$\text{If } A = 1, \text{ then } Y = \overline{1} \text{ or } Y = 0$$

24.22 Combination of Basic Logic Gates

The OR, AND and NOT gates are the three basic circuits that make up all digital circuits. We shall discuss a few combinations of these basic circuits.

(*i*) NAND gate. *When the output of an AND gate is connected to the input of a NOT gate, the resulting gate is called* **NAND gate** as shown in Fig. 24.18 (*i*). The Boolean expression for NAND function is

$$Y = \overline{A.B}$$



(*i*)

INPUTS		OUTPUT	
A	*B*	AND (Y')	NAND (Y)
0	0	0	1
1	0	0	1
0	1	0	1
1	1	1	0

(*ii*)



(*iii*)

Fig. 24.18

To perform the Boolean algebra operation, first the inputs must be ANDed and then the inversion is performed. Fig. 24.18 (*iii*) shows the logic symbol for a NAND gate. The little bubble (small circle) on the right end of the symbol means to invert the AND.

Truth Table for NAND gate. For NAND gate, $Y = \overline{A.B}$. Clearly, the output of a NAND gate is opposite to that of AND gate. This fact is shown in the truth table for NAND gate in Fig. 24.18 (*ii*). Note that truth table for the NAND gate is developed by *inverting the outputs* of the AND gate. *The output of a NAND gate is low when both the inputs are high, otherwise the output is high.*

A NAND gate can be made by connecting a diode AND gate followed by a transistor inverter as shown in Fig. 24.19.

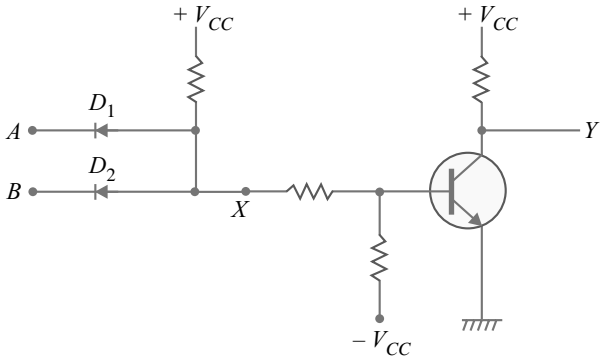


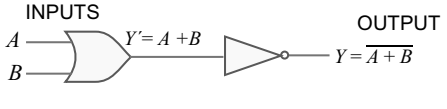
Fig. 24.19

It is clear from the figure that point X would be driven to ground when either of the diodes D_1 or D_2 or both D_1 and D_2 conduct. Under these conditions, the transistor is cut-off and the output Y becomes *high*. The output is *low* only when both input voltages are high (at $+ V_{CC}$) so that X is at $+ V_{CC}$ and transistor operates in saturation.

(*ii*) NOR gate. *When the output of an OR gate is connected to the input of a NOT gate, the resulting gate is called* **NOR gate** as shown in Fig. 24.20 (*i*). The Boolean expression for NOR function is

$$Y = \overline{A + B}$$

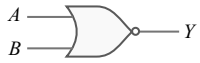
To perform the Boolean algebra operation, first the inputs must be ORed and then the inversion is performed. Fig. 24.20 (*iii*) shows the logic symbol for a NOR gate. The bubble (small circle) at the Y output indicates inversion.



(*i*)

INPUTS		OUTPUT	
A	*B*	OR (*Y*′)	NOR (*Y*)
0	0	0	1
1	0	1	0
0	1	1	0
1	1	1	0

(*ii*)



(*iii*)

Fig. 24.20

Truth table for NOR gate. For NOR gate, $Y = \overline{A + B}$. Clearly, the output of a NOR gate is opposite to that of OR gate. This fact is shown in the truth table for NOR gate in Fig. 24.20 (*ii*). Note that the truth table for NOR gate is developed by inverting the outputs of an OR gate. *The output of a NOR gate is high when both the inputs are low, otherwise the output is low.* A NOR gate can be made by connecting a diode OR gate followed by a transistor inverter as shown in Fig. 24.21.

It is clear from the figure that output *Y* from the NOR gate is *high* only when all the inputs are *low*. If any of the inputs is *high*, the output is *low*.

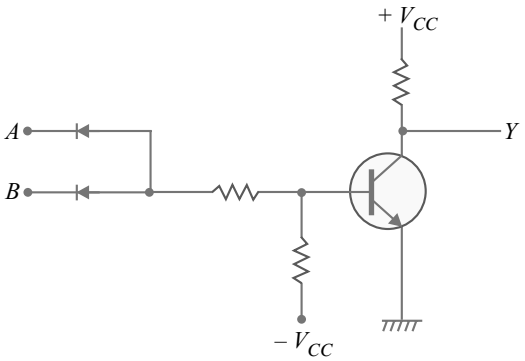
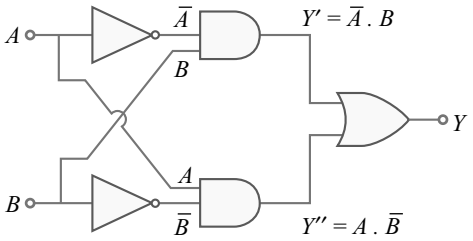


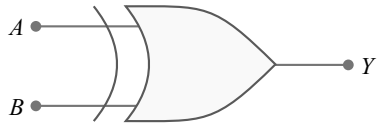
Fig. 24.21

(*iii*) Exclusive OR Gate (XOR Gate). Many digital systems require a circuit which *provides a high output (1) only when one input is high (1) and the other input is low (0).* This job can be done by **exclusive OR** gate usually shortened to XOR gate.

The **exclusive OR gate (or XOR gate)** *gives high output (1) if either input A or input B is high but not when both inputs are high.*



(*i*)



(*ii*)

Fig. 24.22

The XOR gate is obtained by using NOT, AND and OR gates as shown in Fig. 24.22 (*i*). The logic symbol for XOR gate is shown in Fig. 24.22 (*ii*). Note the symbol is similar to that of OR gate except for the additional curved line at the input side. The Boolean expression for XOR gate is :

$$Y = \overline{A}.B + A.\overline{B}$$

or

$$Y = A \oplus B$$

Note the unique XOR symbol with a circle round the + symbol of OR. Read the expression as *A* exclusively ORed with *B* equals output *Y*.

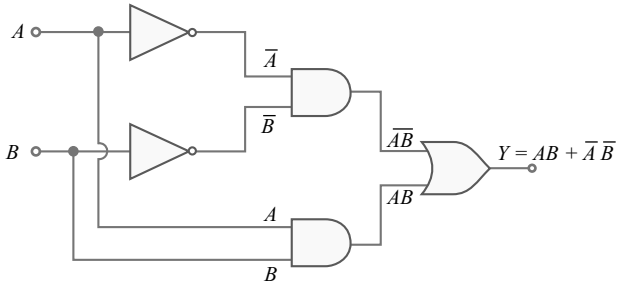
The logic operations in the circuit are as under :

A	B	$\overline{A}$	$\overline{B}$	$\overline{A}.B = Y'$	$A.\overline{B} = Y''$	$Y = Y' + Y''$
0	0	1	1	0	0	0
1	0	0	1	0	1	1
0	1	1	0	1	0	1
1	1	0	0	0	0	0

Note that 0 ANDed with 1 is 0 and 1 ANDed with 1 is 1.

(*iv*) Exclusive NOR gate (XNOR gate). The exclusive NOR gate (usually shortened to XNOR gate) is just an XOR gate followed by an inverter. Therefore, the job of XNOR gate is complement (opposite) of the XOR gate.

The **exclusive NOR gate** (*or* **XNOR gate**) *gives high output (1) only if both inputs are same (0 or 1) and low output (0) if the inputs are different.*



(*i*)

Truth table for XOR and XNOR gate

INPUTS		OUTPUT	
B	*A*	XOR	XNOR
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

(*ii*)

Fig. 24.23

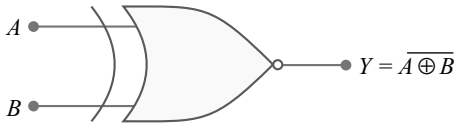
Fig. 24.23 (*i*) shows the circuit of XNOR gate while Fig. 24.23 (*ii*) shows its truth table. The XOR gate output is also shown in the truth table for comparison. Note that output of XNOR gate is complement of the output of XOR gate. The Boolean expression for the XNOR gate is

$$Y = \overline{A \oplus B} = {}^{*}AB + \overline{A}\,\overline{B}$$

The bar over $A \oplus B$ expression tells us that we have inverted the output of the XOR gate. The logic symbol for the XNOR gate is shown in Fig. 24.24. Note that it is the XOR

* Boolean algebra (Refer to Arts. 24.29 and 24.30).

symbol with bubble at the output side and indicates the inversion of the XOR function. The XNOR gate is also called *coincidence gate* because the output is high (1) when its inputs coincide (either 0, 0 or 1, 1).



Symbol for XNOR gate

Fig. 24.24

24.23 Summary of Basic Logic Gates

Fig. 24.25 shows the summary of *seven basic* logic gates. This information should be memorised.

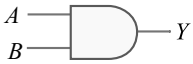
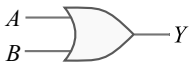
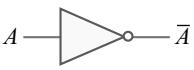
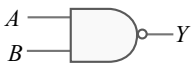
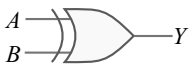
LOGIC FUNCTION	LOGIC SYMBOL	BOOLEAN EXPRESSION	TRUTH TABLE		
			INPUTS		OUTPUT
			B	A	Y
AND		$A \cdot B = Y$	0	0	0
			0	1	0
			1	0	0
			1	1	1
OR		$A + B = Y$	0	0	0
			0	1	1
			1	0	1
			1	1	1
Inverter		$A = \overline{A}$		0	1
				1	0
NAND		$\overline{A \cdot B} = Y$	0	0	1
			0	1	1
			1	0	1
			1	1	0
NOR		$\overline{A + B} = Y$	0	0	1
			0	1	0
			1	0	0
			1	1	0
XOR		$A \oplus B = Y$	0	0	0
			0	1	1
			1	0	1
			1	1	0
XNOR		$\overline{A \oplus B} = Y$	0	0	1
			0	1	0
			1	0	0
			1	1	1

Fig. 24.25

24.24 NAND Gates and NOR Gates as Universal Gates

The NAND gates and NOR gates find wide applications in digital circuits. By connecting NAND gates alone or NOR gates alone, we can produce any of other basic logic gates. In fact, NAND gates and NOR gates are the building blocks in digital circuits. For this reason, NAND gates and NOR gates are called *universal gates*.

24.25 Logic Gates Using NAND Gates Only

By using NAND gates only, we can produce any of the other logic gates. By way of illustration, we now show how NAND gates alone can be wired to produce NOT gate, AND gate and OR gate.

(*i*) NOT gate from NAND gate. When the two inputs of a NAND gate are joined together so that it has one input, the resulting circuit is NOT gate [See Fig. 24.26 (*i*)]. It is because $A = B$ so that $Y = \overline{A.A} = \overline{A} + \overline{A} = \overline{A}$.*



(*i*)

A	*B*	*Y*
0	0	1
1	1	0

(*ii*)

Fig. 24.26

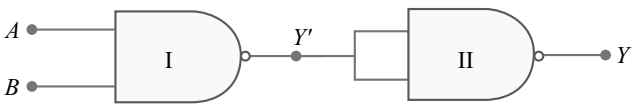
In the truth table of NAND gate if we put $B = A$, we get the truth table of NOT gate as shown in Fig. 24.26 (*ii*). Therefore, the circuit shown in Fig. 24.26 (*i*) works as a NOT gate.

(*ii*) AND gate from NAND gates. For this purpose, we use two NAND gates. If the output of first NAND gate (I) is used as the input to the second NAND gate (II) acting as an inverter or NOT gate (*i.e.,* its two inputs joined), the resulting circuit is the AND gate [See Fig. 24.27 (*i*)]. This is proved as under :

Output of gate I, $Y' = \overline{A.B}$

Output of gate II, $Y = \overline{Y'.Y'} = \overline{\overline{(A.B)}.\overline{(A.B)}} = \overline{\overline{A.B}} + \overline{\overline{A.B}}$ *

$= (A.B) + (A.B) = (A.B)$



(*i*)

A	*B*	*Y'*	*Y*
0	0	1	0
1	0	1	0
0	1	1	0
1	1	0	1

(*ii*)

Fig. 24.27

Since $Y = A.B$ is the Boolean expression for AND gate, the circuit shown in Fig. 24.27 (*i*) is AND gate. Its truth table is shown in Fig. 24.27 (*ii*).

(*iii*) OR gate from NAND gates. For this purpose, we use three NAND gates ; the

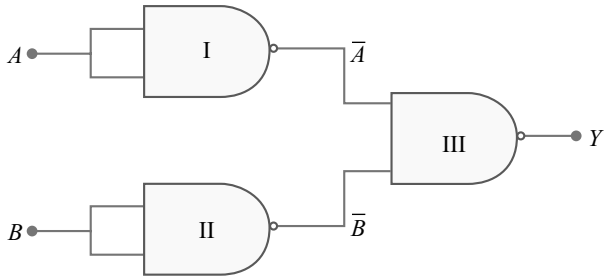
* Boolean algebra (Refer to Arts. 24.29 and 24.30).

first two NAND gates (I and II) are operated as NOT gates and their outputs are fed to the third NAND gate (III). The resulting circuit is OR gate [See Fig. 24.28 (*i*)]. This is proved as under :

$$\text{Output of gate I} = \overline{A.A} = {}^{*}\overline{A} + \overline{A} = \overline{A}$$

$$\text{Output of gate II} = \overline{B.B} = {}^{*}\overline{B} + \overline{B} = \overline{B}$$

$$\text{Output of gate III, } Y = \overline{\overline{A}.\overline{B}} = {}^{*}\overline{\overline{A}} + \overline{\overline{B}} = A + B$$



(*i*)

A	B	$\overline{A}$	$\overline{B}$	Y
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

(*ii*)

Fig. 24.28

Since $Y = A + B$ is the Boolean expression for OR gate, the circuit shown in Fig. 24.28 (*i*) is OR gate. Its truth table is shown in Fig. 24.28 (*ii*).

24.26 Logic Gates Using NOR Gates only

By using NOR gates only, we can produce any of the other logic gates. By way of illustrating, we now shown how NOR gates alone can be wired to produce NOT gate, OR gate and AND gate.

(*i*) NOT gate from NOR gate. Fig. 24.29 (*i*) shows that a NOR gate with its inputs connected together behaves as a NOT gate since $Y = \overline{A + A} = {}^{*}\overline{A}$.

(*ii*) OR gate from NOR gates. Fig. 24.29 (*ii*) shows how two NOR gates are connected to produce an OR gate. The NOR gate II is used as an inverter to change $\overline{A + B}$ to $\overline{\overline{A + B}} = {}^{*}A + B$ which is OR function.

(*iii*) AND gate from NOR gates. Fig. 24.29 (*iii*) shows how three NOR gates are connected to produce the AND gate. Here, NOR gates I and II are used as inverters. The final output is

---

* Boolean algebra (Refer to Arts. 24.29 and 24.30).

$Y = \overline{\overline{A} + \overline{B}}$ = *$A.B$. This is AND function.

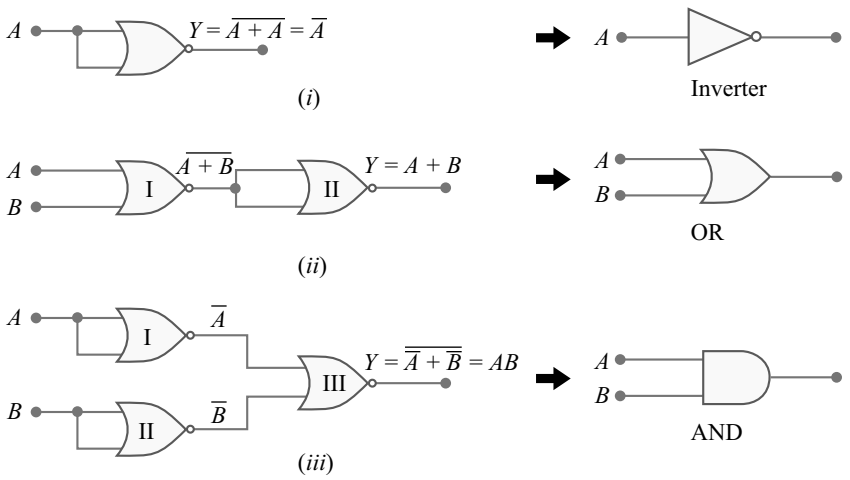
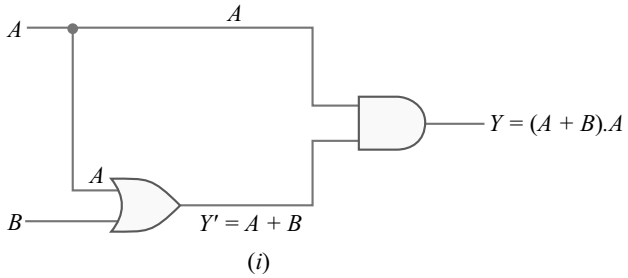


Fig. 24.29

Example 24.24. *Obtain the truth table for the circuit shown in Fig. 24.30 (i).*

Solution. Figure 24.30 (*ii*) shows the truth table for the circuit. The truth table can be obtained very easily if the reader remembers the following simple Boolean operations :



A	B	$Y' = A + B$	$Y = Y' . A$
0	0	0	0
1	0	1	1
0	1	1	0
1	1	1	1

(*ii*)

Fig. 24.30

(*i*) ***ORed with 0 = 0 ; 1 ORed with 1 = 1 ; 1 ORed with 0 = 1.*

(*ii*) *0 !ANDed with 0 = 0 ; 0 ANDed with 1 = 0 ; 1 ANDed with 1 = 1.*

Thus, when $A = 0$ and $B = 0$, then A ORed with $B = 0$ *i.e.,* $Y' = 0$. When $Y'(= 0)$ is ANDed with A (= 0), the result is 0. Again when $A = 1$ and $B = 0$, then A ORed with B is 1 *i.e.,* $Y' = 1$. Now Y' (= 1) ANDed with A (= 1), the result is 1.

---

* Boolean algebra (Refer to Arts. 24.29 and 24.30).

** Note that $A + B$ means A ORed with B.

! Note that $A.B$ means A ANDed with B.

Example 24.25. *Obtain the truth table for the circuit shown in Fig. 24.31.*

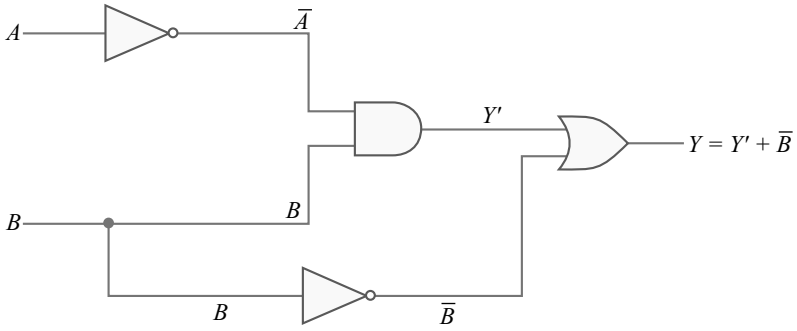


Fig. 24.31

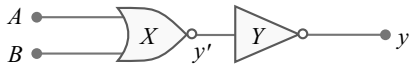
Solution. The truth table for the circuit is shown below :

A	B	$\overline{A}$	$Y' = \overline{A}.B$	$\overline{B}$	$Y = Y' + \overline{B}$
0	0	1	0	1	1
1	0	0	0	1	1
0	1	1	1	0	1
1	1	0	0	0	0

(*i*) When $A = 0$ and $B = 0$, then $\overline{A} = 1$. Now Y' is equal to $\overline{A}$ (= 1) ANDed with B (= 0). The result is 0. Then Y' (= 0) ORed with $\overline{B}$ (= 1) is 1 *i.e.,* $Y = 1$.

(*ii*) When $A = 1$ and $B = 0$, then $\overline{A} = 0$. Now Y' is equal to $\overline{A}$ (= 0) ANDed with B (= 0) and the result is 0 *i.e.,* $Y' = 0$. Then Y' (= 0) ORed with $\overline{B}$ (= 1) is 1 *i.e.,* $Y = 1$. The reader can proceed in a similar way to find the other output values.

Example 24.26. *Identify the logic gates marked X, Y in Fig. 24.32. Write down the output at y, when A = 1, B = 1 and A = 0, B = 0.*



A	B	y'	y
1	1	0	1
0	0	1	0

Fig. 24.32

Solution. The logic gate marked as X is NOR gate while the gate marked as Y is NOT gate. It is clear that :

$$y' = \overline{A + B} \text{ and } y = \overline{y}'$$

Therefore, the output at y will be as shown in the truth table.

Example 24.27. *Sketch the output (Y) waveform from a NOR gate for the input A and B shown in Fig. 24.33.*

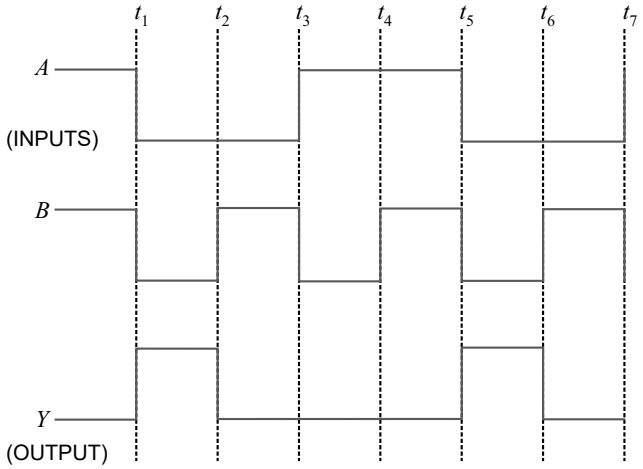


Fig. 24.33

Solution. The output from a NOR gate is high (1) only when all the inputs are low (0). If any of the inputs is high (1), the output is low (0).

For $t < t_1$; $A = 1$, $B = 1$; Hence $Y = 0$.

For t_1 to t_2 ; $A = 0$, $B = 0$; Hence $Y = 1$.

For t_2 to t_3 ; $A = 0$, $B = 1$; Hence $Y = 0$.

For t_3 to t_4 ; $A = 1$, $B = 0$; Hence $Y = 0$.

For t_4 to t_5 ; $A = 1$, $B = 1$; Hence $Y = 0$.

For t_5 to t_6 ; $A = 0$, $B = 0$; Hence $Y = 1$.

For t_6 to t_7 ; $A = 0$, $B = 1$; Hence $Y = 0$.

Example 24.28. *Sketch the output (Y) waveform from a NOT gate having input A as shown in Fig. 24.34 (i).*

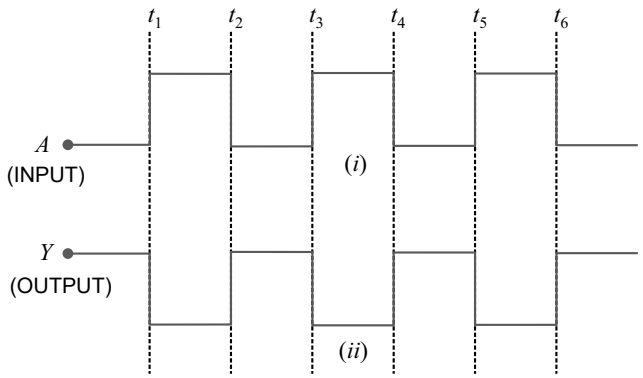


Fig. 24.34

Solution. The output (Y) of a NOT gate is high (1) when input is low (0) and *vice-versa* ($Y = \overline{A}$).

For $t \leq t_1$; $A = 0$, Hence $Y = 1$.

For t_1 to t_2 ; $A = 1$, Hence $Y = 0$.

For t_2 to t_3 ; $A = 0$, Hence $Y = 1$.

For t_3 to t_4 ; $A = 1$, Hence $Y = 0$.

For t_4 to t_5 ; $A = 0$, Hence $Y = 1$.

For t_5 to t_6 ; $A = 1$, Hence $Y = 0$.

For $t > t_6$; $A = 0$, Hence $Y = 1$.

The waveform of the output Y for the given waveform will be as shown in Fig. 24.34 (*ii*).

Example 24.29. *Realise XOR gate using NAND gates.*

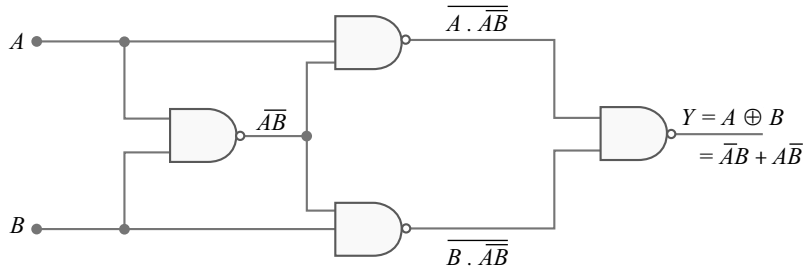


Fig. 24.35

Solution. The XOR gate can be produced by using four NAND gates as shown in Fig. 24.35.

24.27 Encoders and Decoders

A digital circuit can process numbers in binary form. However, most of the information we handle is in decimal form. Therefore, a digital machine must perform the following functions :

(*i*) Convert the information from decimal to digital (binary) form.

(*ii*) Process the digital information.

(*iii*) Convert the digital output back to decimal form.

The circuit that converts decimal form to digital (binary) form is called **encoder** and the circuit that converts digital form to decimal form is called **decoder.** Fig. 24.36 shows encoding and decoding in a digital calculator. Here the input is the decimal number 5 punched in at the keyboard. The encoder changes the decimal number 5 to the digital form as the binary digit 0101. The central processing unit (CPU) contains digital logic circuits for necessary calculations. Here all operations are carried out in binary form. The output of CPU is fed to the decoder which changes the binary signal back to the decimal form. The output display is in the decimal form, showing the original number 5.

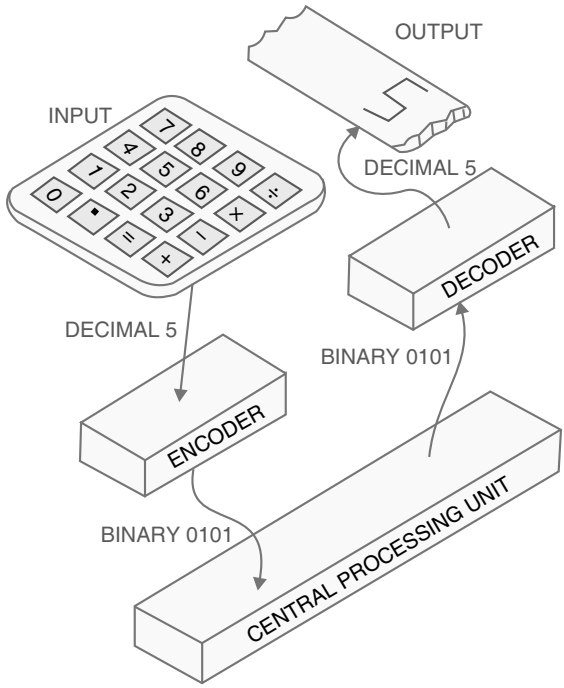


Fig. 24.36

24.28 Advantages and Disadvantages of Digital Electronics

The world of electronics can be classified as either digital or analog circuits. An increasing majority of applications in electronics use digital techniques to perform operations that were once performed using analog methods. It is worthwhile to give advantages and disadvantages of digital electronics.

Advantages. The chief reasons for the shift to digital technology are :

(*i*) *Digital systems are generally easier to design.* It is because the circuits that are used are *switching circuits* where *exact* values of voltages or currents are not important, only the range (HIGH or LOW) in which they fall is important.

(*ii*) *Digital circuits provide greater accuracy and precision.* It is because digital circuits can handle as many digits of precision as you need simply by adding more switching circuits. In analog systems, precision is usually limited to three or four digits because the values of voltage and current are directly dependent on the circuit components.

(*iii*) *Digital circuits are less affected by noise.* Spurious fluctuations in voltage (noise) are not as critical in digital systems as in analog systems. It is because in a digital circuit, the exact value of a voltage is not important as long as the noise is not large enough to prevent us from distinguishing a HIGH from a LOW.

(*iv*) *More digital circuitry can be fabricated on IC chips.* Analog system uses such devices (high-value capacitors, inductors, transformers) that cannot be economically integrated. For this reason, analog systems cannot achieve the same degree of integration as digital circuits.

(*v*) Information storage is easy with digital circuits.

Disadvantages. (*i*) The real world is mainly analog. However, the digital circuits can handle only digital signals. This necessitates encoders and decoders which increase the cost of the equipment.

(*ii*) There are situations where using only analog techniques is simpler and more economical. For example, the process of signal amplification is most easily accomplished using analog circuitry.

However, advantages of digital techniques outweigh the disadvantages. For this reason, we are fast switching to digital techniques.

24.29 Boolean Algebra

Digital circuits perform the binary arithmetic operations with binary digits 1 and 0. These operations are called logic functions or logical operations. *The algebra used to symbolically describe logic functions is called* **Boolean algebra.** Boolean algebra is a set of rules and theorems by which logical operations can be expressed symbolically in equation form and be manipulated mathematically. As with the ordinary algebra, the *letters of alphabet (*e.g. A, B, C etc.*) can be used to represent the variables. Boolean algebra differs from ordinary algebra in that Boolean constants and variables can have only two values ; 0 and 1. There are four connecting symbols used in Boolean algebra *viz*.

(*i*) equals sign (=) (*ii*) plus sign (+)

(*iii*) multiply sign (·) (*iv*) bar (–)

(*i*) **Equals sign (=).** The equals sign in Boolean algebra refers to the standard mathematical equality. In other words, the logical value on one side of the sign is identical to the logical value on the other side of the sign. Suppose we are given two logical variables such that $A = B$. Then if $A = 1$, then $B = 1$ and if $A = 0$, then $B = 0$.

(*ii*) **Plus sign (+).** The plus sign in Boolean algebra refers to the logical *OR operation*. Thus, when the statement $A + B = 1$ appears in Boolean algebra, it means A ORed with B equals 1. Consequently, either $A = 1$ or $B = 1$ or both equal 1.

(*iii*) **Multiply sign (·).** The multiply sign in Boolean algebra refers to *AND operation*. Thus, when the statement $A \cdot B = 1$ appears in Boolean algebra, it means A ANDed with B equals 1. Consequently, $A = 1$ and $B = 1$. The function $A \cdot B$ is often written as AB, omitting the dot for convenience.

(*iv*) **Bar sign (–).** The bar sign in Boolean algebra refers to *NOT operation*. The NOT has the effect of inverting (complementing) the logical value. Thus, if $A = 1$, then $\overline{A} = 0$.

24.30 Boolean Theorems

We now discuss the basic Boolean theorems that are useful in manipulating and simplifying Boolean expressions. For convenience, we divide the theorems into two groups :

(*i*) Single variable theorems

(*ii*) Multivariable theorems

* For example, A might represent a certain digital circuit input or output and at any time, we must have either $A = 0$ or $A = 1$.

(*i*) Single variable theorems. These theorems refer to the condition when only one input to the logic gate is variable. Table 24.1 gives single variable Boolean theorems.

Table 24.1

Theorem 1 :	$A + 0 = A$
Theorem 2 :	$A \cdot 1 = A$
Theorem 3 :	$A + \overline{A} = 1$
Theorem 4 :	$A \cdot \overline{A} = 0$
Theorem 5 :	$A + A = A$
Theorem 6 :	$A \cdot A = A$
Theorem 7 :	$A + 1 = 1$
Theorem 8 :	$A \cdot 0 = 0$
Theorem 9 :	$\overline{\overline{A}} = A$

Theorem 1. ($A + 0 = A$). This theorem can be verified by ORing a variable A with a 0 and is illustrated in Fig. 24.37. Here one input to OR gate is always 0 and the other input A can be a value 1 or 0. When A is at 1, the output is 1 which is equal to A. When A is at 0, the output is 0 which is also equal to A (= 0). Therefore, a variable ORed with 0 is equal to the value of the variable. This is easy to remember since 0 added to anything does not effect the value of the variable, either in regular addition or OR addition.

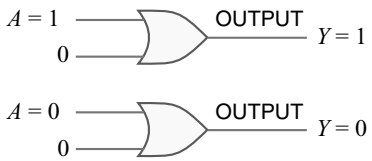


Fig. 24.37

Theorem 2. ($A \cdot 1 = A$). This theorem can be verified by ANDing a variable A with a 1 and is illustrated in Fig. 24.38. Here one input to AND gate is always 1 and the other can be a value 1 or 0. If A is 1, the output of the AND gate is 1 because both the inputs are now 1's. If A is 0, the output of the AND gate is a 0. Therefore, a variable ANDed with a 1 is equal to the value of the variable ($A \cdot 1 = A$). This is easy to remember because AND operation is just like ordinary multiplication.

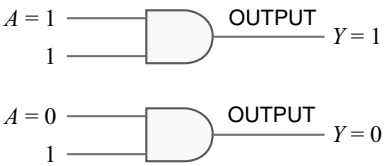


Fig. 24.38

Theorem 3. ($A + \overline{A} = 1$). This theorem can be easily explained. If a variable A and its complement ($\overline{A}$) are ORed, the result is always 1. If A is a 0, then $0 + \overline{0} = 0 + 1 = 1$. If A is a 1, then $1 + \overline{1} = 1 + 0 = 1$. Fig. 24.39 illustrates this theorem.

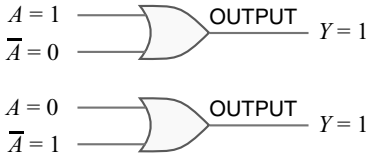


Fig. 24.39

Theorem 4. ($A \cdot \overline{A} = 0$). This theorem states that if a variable A is ANDed with its complement, the result is zero. This is readily apparent because either A or $\overline{A}$ will always be 0. Therefore, when one of the inputs to an AND gate is 0, the output is always 0. Fig. 24.40 illustrates this theorem.

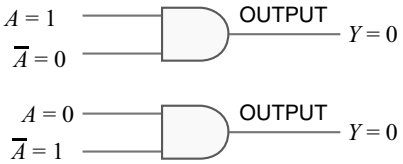


Fig. 24.40

Theorem 5. ($A + A = A$). This theorem states that when a variable A is ORed with itself, the output is equal to the variable. Thus, if A is a 0, then $0 + 0 = 0$ and if A is a 1, then $1 + 1 = 1$. Fig. 24.41 illustrates this theorem.

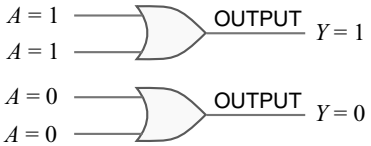


Fig. 24.41

Theorem 6. ($A \cdot A = A$). This theorem states that if a variable A is ANDed with itself, the result is equal to the variable. For example, if $A = 0$, then $0 \cdot 0 = 0$ and if $A = 1$, then $1 \cdot 1 = 1$. For either case, the output of an AND gate is equal to the value of the input variable A. Fig. 24.42 illustrates this theorem.

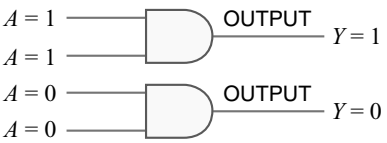


Fig. 24.42

Theorem 7. ($A + 1 = 1$). This theorem states that when a variable A is ORed with 1, the outuput is always equal to 1. Fig. 24.43 illustrates this theorem. One input to an OR gate is always 1 and the other input A can be either 1 or 0. Now 1 on an input to OR gate produces 1 on the output regardless of the value of the variable on the other input.

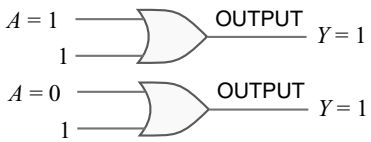


Fig. 24.43

Theorem 8. ($A \cdot 0 = 0$). This theorem states that variable A ANDed with 0 always produces 0. Recall that any time one input to an AND gate is 0, the output is 0 regardless of the value of the variable A on the other input. This theorem is illustrated in Fig. 24.44.

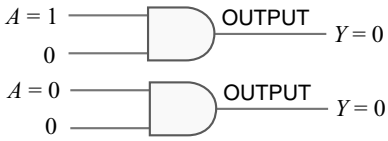


Fig. 24.44

Theorem 9. ($\overline{\overline{A}} = A$). This theorem states that if a variable A is complemented twice, the result is the variable itself. Starting with A and inverting (complementing) it once gives $\overline{A}$. Inverting it once more gives A — the original value. This theorem is illustrated in Fig. 24.45.

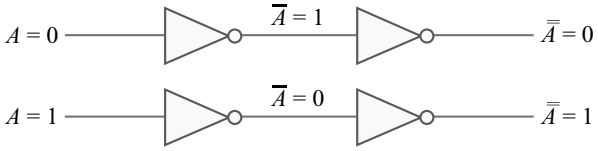


Fig. 24.45

Duality Principle. Before moving to multivariable theorems, this would be the right place to mention an important property of Boolean algebra called *duality principle*. It is stated below :

The duality principle states that a Boolean expression remains valid if operators OR and AND are interchanged and 1's and 0's in the expression are also interchanged.

In order to understand this principle, consider the Boolean Theorem 1 *viz.*

$$A + 0 = A$$

According to duality principle, this Boolean expression remains valid if OR function is replaced by AND function and 0 by 1. In that case, the Boolean expression becomes :

$$A \cdot 1 = A$$

Note that this is Boolean Theorem No. 2. Therefore, Boolean Theorem 2 is dual of Boolean Theorem 1 and *vice-versa*. Applying duality principle, Theorem 4 is dual of Theorem 3 and *vice-versa,* Theorem 6 is dual of Theorem 5 and *vice-versa*, Theorem 8 is dual of Theorem 7 and *vice-versa*. To apply duality principle to a Boolean expression, we simply interchange OR and AND operator and replace 1's by 0's and 0's by 1's.

(*ii*) Multivariable theorems. These theorems refer to the condition when more than one input to the logic gate are variable. Table 24.2 gives multivariable Boolean theorems.

Table 24.2

Theorem 10 :	$A + B = B + A$	*Commutative Law*
Theorem 11 :	$A \cdot B = B \cdot A$	
Theorem 12 :	$A + (B + C) = (A + B) + C$	*Associative Law*
Theorem 13 :	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	

Theorem 14 : $A \cdot (B + C) = A \cdot B + A \cdot C$

Theorem 15 : $(A + B) \cdot (C + D) = A \cdot C + B \cdot C + A \cdot D + B \cdot D$ } *Distributive Law*

Theorem 16 : $A + A \cdot B = A$

Theorem 17 : $\overline{(A + B)} = \overline{A} \cdot \overline{B}$ } *De Morgan's Theorems*

Theorem 18 : $\overline{(A \cdot B)} = \overline{A} + \overline{B}$

The following points may be noted about these theorems :

(*a*) Theorems 10 and 11 obey **commutative law**. *This law states that the order in which the variables are ORed or ANDed makes no difference.*



Fig. 24.46



Fig. 24.47

Figure 24.46 illustrates the commutative law as applied to the OR gate while Fig. 24.47 illustrates the commutative law as applied to an AND gate.

(*b*) Theorems 12 and 13 obey **associative law.** *This law states that in the ORing or ANDing of several variables, the result is the same regardless of the grouping of the variables.*

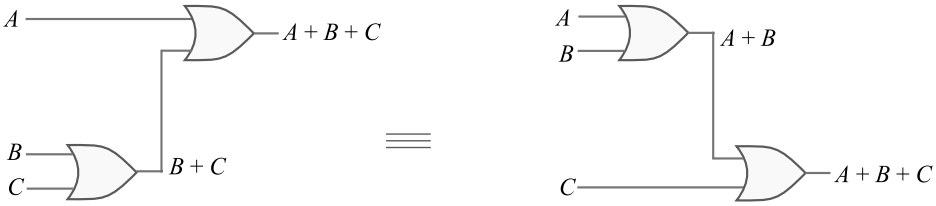


Fig. 24.48



Fig. 24.49

Figure 24.48 illustrates the associative law as applied to the OR gate while Fig. 24.49 illustrates the associative law as applied to an AND gate.

(*c*) Theorems 14 and 15 obey **distributive law**. *This law states that a Boolean expression can be expanded by multiplying term-by-term just the same as in ordinary algebra.*



Fig. 24.50

Fig. 24.50 illustrates the distributive law in terms of gate implementation.

(*d*) We will prove Theorem 16 by factoring and using Theorems 2, 7, 10 and 14.

$$\begin{aligned} A + A \cdot B &= A \cdot 1 + A \cdot B && \text{...Theorem 2} \\ &= A \cdot (1 + B) && \text{...Theorem 14} \\ &= A \cdot (B + 1) && \text{...Theorem 10} \\ &= A \cdot 1 && \text{...Theorem 7} \\ &= A && \text{...Theorem 2} \end{aligned}$$

(*e*) Theorems 17 and 18 are the two most important theorems of Boolean algebra and were contributed by the great mathematician named *De Morgan*. Therefore, these theorems are called De Morgan's theorems.

24.31 De Morgan's Theorems

De Morgan's theorems are extremely useful in simplifying expressions in which a product or sum of variables is inverted. The two theorems are :

(*i*) $\overline{(A+B)} = \overline{A} \cdot \overline{B}$

(*ii*) $\overline{(A.B)} = \overline{A} + \overline{B}$

(*i*) The first De Morgan's theorem may be stated as under :

When the OR sum of two variables is inverted, this is equal to inverting each variable individually and then ANDing these inverted variables i.e.,

$$\overline{(A+B)} = \overline{A \cdot B}$$

In this expression, A and B are the two variables. The L.H.S. is the complement of the OR sum of the two variables. The R.H.S. is the AND product of individual inverted variables.

(*ii*) The second De Morgan's theorem may be stated as under :

When the AND product of two variables is inverted, this is equal to inverting each variable individually and then ORing them i.e.,

$$\overline{(A \cdot B)} = \overline{A} + \overline{B}$$

In this expression, A and B are the two variables. The L.H.S. is the complement of the AND product of the two variables. The R.H.S. is the OR sum of the individual inverted variables.

24.32 Operator Precedence

The operator precedence for evaluating Boolean expression is (*i*) parenthesis (*ii*) NOT (*iii*) AND and (*iv*) OR. In other words, the expression inside the parenthesis must be evaluated

before all other operations. The next operation that holds precedence is the complement, then follows the AND and finally the OR. For example, consider the Boolean expression :

$$A + \overline{B} \,.\, (C + D)$$

The sequence of operations will be :

(*i*) The expression inside the parenthesis (*i.e.,* $C + D$) will be evaluated first.

(*ii*) Then $\overline{B}$ will be evaluated.

(*iii*) Then the results of the two (*i.e.,* $\overline{B}$ and $C + D$) will be ANDed.

(*iv*) Finally, the result of the product will be ORed with A.

Example 24.30. *Using Boolean algebraic techniques, simplify the following expression* :

$$Y = A \cdot B \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot B \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot B \cdot C \cdot \overline{D} + A \cdot B \cdot C \cdot \overline{D}$$

Solution. $$Y = A \cdot B \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot B \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot B \cdot C \cdot \overline{D} + A \cdot B \cdot C \cdot \overline{D} \quad ...(i)$$

Step 1 : Take out the common factors as below :

$$Y = B\,\overline{C}\,\overline{D}\,(A + \overline{A}) + B\,C\,\overline{D}\,(A + \overline{A})$$

Step 2 : Apply Theorem 3 $(A + \overline{A} = 1)$:

$$Y = B\,\overline{C}\,\overline{D} + B\,C\,\overline{D}$$

Step 3 : Again factorise :

$$Y = B\,\overline{D}\,(C + \overline{C})$$

Step 4 : Apply Theorem 3 $(C + \overline{C} = 1)$:

$$Y = B\,\overline{D} \cdot 1 = B\overline{D}$$

This is the simplified form of exp. (*i*).

Example 24.31. *Using Boolean techniques, simplify the following expression* :

$$Y = AB + A\,(B + C) + B\,(B + C)$$

Solution. $$Y = AB + A\,(B + C) + B\,(B + C) \quad ...(i)$$

Step 1 : Apply Theorem 14 (distributive law) to second and third terms :

$$Y = AB + AB + AC + BB + BC$$

Step 2 : Apply Theorem 6 $(B \cdot B = B)$:

$$Y = AB + AB + AC + B + BC$$

Step 3 : Apply Theorem 5 $(AB + AB = AB)$:

$$Y = AB + AC + B + BC$$

Step 4 : Factor B out of last 2 terms :

$$Y = AB + AC + B\,(1 + C)$$

Step 5 : Apply commutative law and Theorem 7 $(1 + C = C + 1 = 1)$:

$$Y = AB + AC + B \cdot 1$$

Step 6 : Apply Theorem 2 $(B \cdot 1 = B)$:

$$Y = AB + AC + B$$

Step 7 : Factor B out of first and third terms :

$$Y = B\,(A + 1) + AC$$

Step 8 : Apply Theorem 7 ($A + 1 = 1$) :

$$Y = B \cdot 1 + AC$$

Step 9 : Apply Theorem 2 ($B \cdot 1 = B$) :

$$\boldsymbol{Y = B + AC}$$

This is the simplified form of exp. (*i*).

Example 24.32. *Simplify the following Boolean expressions to a minimum number of literals* :

(*i*) $Y = A + \overline{A}B$ (*ii*) $Y = AB + \overline{A}C + BC$

Solution. (*i*)

$$\begin{aligned} Y &= A + \overline{A}B \\ &= A + AB + \overline{A}B \quad [\because A = A + AB \text{ from Theorem 16}] \\ &= A + B(A + \overline{A}) \\ &= A + B \quad [\because A + \overline{A} = 1 \text{ from Theorem 3}] \end{aligned}$$

$$\therefore \quad \boldsymbol{Y = A + B}$$

(*ii*)

$$\begin{aligned} Y &= AB + \overline{A}C + BC \\ &= AB + \overline{A}C + BC \cdot (A + \overline{A}) \\ &= AB + \overline{A}C + ABC + \overline{A}BC \\ &= AB(1 + C) + \overline{A}C\,(1 + B) \\ &= AB + \overline{A}C \end{aligned}$$

$$\therefore \quad \boldsymbol{Y = AB + \overline{A}C}$$

Example 24.33. *Determine output expression for the circuit shown below and simplify it using De Morgan's theorem.*

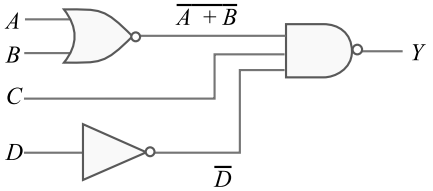


Fig. 24.51

Solution. The output expression for the circuit shown above is :

$$Y = [\overline{(\overline{A + B}) \cdot C \cdot \overline{D}}]$$

Using De Morgan's theorem :

$$Y = (A + B) + \overline{C} + D$$

$$\therefore \quad \boldsymbol{Y = A + B + \overline{C} + D}$$

Example 24.34. *Find the complement of the expressions given below* :

(*i*) $Y = AB\overline{C} + A\,\overline{BC}$

(*ii*) $Y = \overline{A} + (B\overline{C} + \overline{B}C)$

Solution. (*i*)

$$Y = AB\overline{C} + A\,\overline{BC}$$

$$\overline{Y} = (\overline{AB\overline{C} + A\,\overline{BC}})$$

Applying De Morgan's theorem :

$$\overline{Y} = \overline{(A\overline{BC})} \cdot \overline{(A\overline{\overline{BC}})}$$

Again applying De Morgan's theorem to the each expression inside the brackets :

$$\overline{Y} = (\overline{A} + \overline{B} + C) \cdot (\overline{A} + B + C)$$

(*ii*) $$Y = \overline{A}(B\overline{C} + \overline{B}C)$$

$$\overline{Y} = \overline{\overline{A}(B\overline{C} + \overline{B}C)}$$

Applying De Morgan's theorem :

$$\overline{Y} = A + \overline{(B\overline{C} + \overline{B}C)}$$

Again applying De Morgan's theorem to the expression inside the bracket :

$$\overline{Y} = A + \overline{(B\overline{C})} \cdot \overline{(\overline{B}C)}$$

Applying De Morgan's theorem for the third time, we get,

$$\overline{Y} = A + (\overline{B} + C) \cdot (B + \overline{C})$$

or $$\overline{Y} = A + \overline{BC} + BC$$

Example 24.35. *Simplify the following Boolean expressions* :

(*i*) $Y = (A + B + C) \cdot (A + B)$

(*ii*) $Y = AB + ABC + AB\overline{C}$

(*iii*) $Y = 1 + A\,(B \cdot \overline{C} + BC + \overline{B}\,\overline{C}) + A\overline{B}C + AC$

(*iv*) $Y = \overline{(A + \overline{B} + C) + (B + \overline{C})}$

Solution. (*i*)
$$Y = (A + B + C) \cdot (A + B)$$
$$= A \cdot A + A \cdot B + B \cdot A + B \cdot B + C \cdot A + C \cdot B$$

Using $A \cdot A = A$, we get,

$$Y = A + AB + AB + B + AC + BC$$
$$= A + AB + B + AC + BC \qquad [\because AB + AB = AB]$$
$$= A + B + AC + BC \qquad [\because A + AB = A]$$
$$= A(1 + C) + B(1 + C)$$
$$= A \cdot 1 + B \cdot 1 \qquad [\because 1 + C = 1]$$

$\therefore$ $$Y = A + B$$

(*ii*)
$$Y = AB + ABC + AB\overline{C}$$
$$= AB + AB(C + \overline{C})$$
$$= AB + AB \qquad [\because C + \overline{C} = 1]$$

$\therefore$ $$Y = AB$$

(*iii*)
$$Y = 1 + A\,(B \cdot \overline{C} + BC + \overline{B}\,\overline{C}) + A\overline{B}C + AC$$

Using $1 + A = 1$, we get,

$$Y = 1 + A\overline{B}C + AC \qquad [\because 1 + A(B\overline{C} + BC + \overline{BC}) = 1]$$
$$= 1 + AC$$

$\therefore \qquad Y = 1$

Thus, because of the first term Y reduces to 1. Therefore, any Boolean expression ORed with 1, results in 1.

(iv) $\qquad Y = \overline{(A + \overline{B} + C) + (B + \overline{C})}$

Applying De Morgan's theorem :

$$Y = \overline{(A + \overline{B} + C)} \cdot \overline{(B + \overline{C})}$$

Again applying De Morgan's theorem :

$$Y = (\overline{A} \cdot B \cdot \overline{C}) \cdot (\overline{B} \cdot C) = 0 \quad [\because B \cdot \overline{B} = 0,\ C \cdot \overline{C} = 0]$$

Example 24.36. *Simplify the following Boolean expression* :

$$Y = A\,\overline{B}\,D + A\,\overline{B}\,\overline{D}$$

Solution. $\qquad Y = A\,\overline{B}\,D + A\,\overline{B}\,\overline{D}$

Factoring out the common variables $A\,\overline{B}$ (using Theorem 14), we get,

$$Y = A\,\overline{B}\,(D + \overline{D})$$

Using Theorem 3, $D + \overline{D} = 1$.

$\therefore \qquad Y = A\,\overline{B} \cdot 1$

Using Theorem 2, we get,

$$Y = A\,\overline{B}$$

Example 24.37. *Simplify the following Boolean expression* :

$$Y = (\overline{A} + B)\,(A + B)$$

Solution. $\qquad Y = (\overline{A} + B)\,(A + B)$

The expression can be expanded by multiplying out the terms [Theorem 15].

$$Y = \overline{A} \cdot A + \overline{A} \cdot B + B \cdot A + B \cdot B$$

Using Theorem 4, $\overline{A} \cdot A = 0$. Also $B \,.\, B = B$ [Theorem 6].

$\therefore \qquad Y = 0 + \overline{A} \cdot B + B \cdot A + B$

$$= \overline{A} \cdot B + A\,B + B$$

Factoring out the variable B [Theorem 14], we have,

$$Y = B\,(\overline{A} + A + 1)$$

Using Theorem 7, $A + 1 = 1$.

$\therefore \qquad Y = B\,(\overline{A} + 1)$

Again using Theorem 7, $\overline{A} + 1 = 1$.

$\therefore \qquad Y = B \cdot 1$

Finally, using Theorem 2, we have,

$$Y = B$$

24.33 Combinational and Sequential Logic Circuits

We classify logic circuits in two groups *viz*.

(*i*) Combinational logic circuits

(*ii*) Sequential logic circuits

(*i*) Combinational logic circuits. We connect two or more than two logic gates (*e.g.* OR gate, AND gate, NOT gate *etc.*) to perform such functions as binary addition, subtraction *etc*. When such a logic circuit has no storage capability, it is called a combinational logic circuit. This type of circuit *combines* the input variables in such a way that output is always dependent on the combination of inputs.

(*ii*) Sequential logic circuits. In this type of logic circuit, the logic gates are connected in such a way that binary information can be stored. The output of a sequential logic circuit depends on both past and present input conditions. A good example of a sequential system is a telephone. When the last number is dialed, the party reached (output) is dependent on this last number (present value of input) and the past history of the system (previously dialed numbers).

24.34 Boolean Expressions for Combinational Logic Circuits

A combinational logic circuit (or digital circuit) often consists of several different logic gates, interconnected in such a way as to perform a specific logic function. By using the laws, theorems and techniques of Boolean algebra, we can find the Boolean expression for any combinational logic circuit. Let us find Boolean expression for some logic circuits.

(*i*) Fig. 24.52 shows a logic circuit. It consists of two AND gates and one OR gate. Each of the three gates has two input variables. Because gate G_1 is an AND gate and its two inputs are A and B, its output is AB. The gate G_2 is also an AND gate and its two inputs are C and D so that its output is CD. The gate G_3 is an OR gate so that its output is the ORing of AB and CD, producing $AB + CD$. Therefore, the Boolean expression for the output of this logic circuit is

$$Y = AB + CD$$

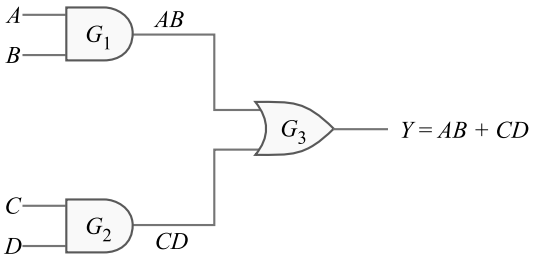


Fig. 24.52

(*ii*) Fig. 24.53 shows a logic circuit consisting of two AND gates and one OR gate. There are four (A, B, C and D) input variables.

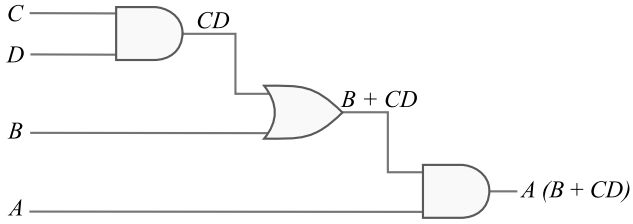


Fig. 24.53

The variable C is ANDed with D and its output is CD. Then CD is ORed with B, giving an output $(B + CD)$. This sum is then ANDed with A, resulting in the following Boolean expression :

$$Y = A\ (B + CD)$$

(***iii***) Fig. 24.54 shows a logic circuit consisting of three AND gates and one OR gate. There are six input variables *viz*. A, B, C, D, E and F.

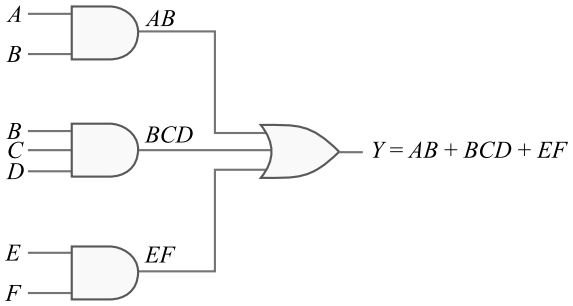


Fig. 24.54

The variables A and B are ANDed to produce AB, B, C and D are ANDed to produce BCD and E and F are ANDed to produce EF. The input to OR gate is AB, BCD and EF so the output of this gate is the ORing of the inputs *i.e.,*

$$Y = AB + BCD + EF$$

24.35 AND and OR Operations in Boolean Expression

It is very important to interpret AND and OR operations in the Boolean expression. Sometimes, there may be confusion as to which operation (AND or OR) in a Boolean expression is to be performed first. For example, the Boolean expression *$A \cdot B + C$ can be interpreted in two different ways *viz*.

(***i***) $A \cdot B$ is ORed with C *or* (***ii***) A is ANDed with the term $B + C$

In order to avoid this confusion, it will be understood that if Boolean expression contains both AND and OR operations, the AND operations are performed first unless there are brackets in the expression in which case the operation inside the brackets is to be performed first. This is the same rule that is used in ordinary algebra to determine the order of operations. Let us illustrate this important point with examples.

(***a***) Consider the Boolean expression : $Y = A \cdot B + C$. Since there are no brackets in this expression, AND operation is to be performed first. In other words, first A and B

* Recall that it indicates A ANDed with B and can be expressed as $A \cdot B$ or AB.

are ANDed to produce $A \cdot B$ and then $A \cdot B$ is ORed with C to give the final function : $Y = A \cdot B + C$. The logic circuit representing this Boolean expression is shown in Fig. 24.55.

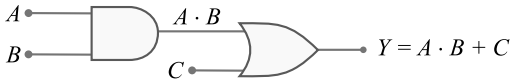


Fig. 24.55

(*b*) Consider the Boolean expression : $Y = (A + B) \cdot C$. Since $A + B$ is in the bracket, the OR operation is to be performed first. In other words, A is ORed with B to give $(A + B)$ and then $(A + B)$ is ANDed with C to produce $Y = (A + B) \cdot C$. The logic circuit representing this Boolean expression is shown in Fig. 24.56.

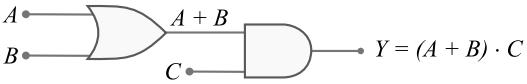


Fig. 24.56

Example 24.38. *Write the Boolean expression for the digital circuit shown in Fig. 24.57.*

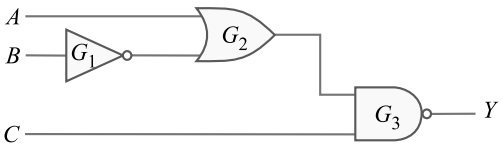


Fig. 24.57

Solution. The output of G_2 is $A + \overline{B}$. This output of G_2 is NANDed with C to yield $Y = \overline{(A + \overline{B})\, C}$. Note that bracket around $(A + \overline{B})$ is required otherwise the expression may be interpreted that A is NORed with $\overline{B}\, C$.

Example 24.39. *Write the Boolean expression for the logic circuit (digital circuit) shown in Fig. 24.58.*

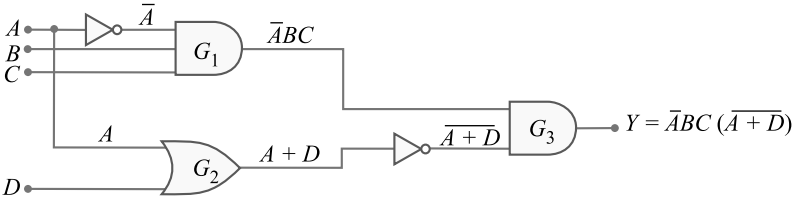


Fig. 24.58

Solution. There are two AND gates (*viz.* G_1 and G_3) and one OR gate (G_2). Also the circuit has two inverters. The input A is fed to G_1 through the inverter. Therefore, $\overline{A}$, B and C are ANDed by G_1 to produce an output of $\overline{A}\, BC$. The input A is ORed with D by G_2 to produce an output of $(A + D)$. The inputs $\overline{A}BC$ and $\overline{(A + D)}$ to G_3 are ANDed to produce the final output :

$$Y = \overline{A}BC\ \overline{(A + D)}$$

Example 24.40. *Write the Boolean expression for the digital circuit shown in Fig. 24.59.*

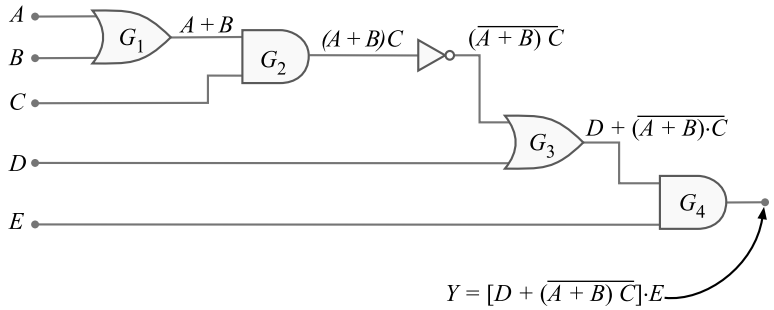


Fig. 24.59

Solution. The output of G_1 is $A + B$. Then $(A + B)$ and C are ANDed to produce an output of $(A + B)\ C$. It is easy to see $\overline{(A + B)\ C}$ and D are ORed by G_3 to produce an output of $D + \overline{(A + B) \cdot C}$. Finally, $D + \overline{(A + B) \cdot C}$ and E are ANDed by G_4 to produce the final output Y given by ;

$$Y = [D + \overline{(A + B)\ C}\ ] \cdot E$$

Example 24.41. *Illustrate the commutative law of (i) addition and (ii) multiplication of two variables.*

Solution.

(*i*) The commutative law of addition states that the order in which the variables are ORed makes no difference. This law for the addition of two variables A and B can be written as :

$$A + B = B + A$$



Fig. 24.60

Fig. 24.60 illustrates the commutative law as applied to the OR gate.

(*ii*) The commutative law of multiplication states that the order in which the variables are ANDed makes no difference. This law for the multiplication of two variables A and B can be written as :

$$AB = BA$$



Fig. 24.61

Fig. 24.61 illustrates the commutative law as applied to the AND gate.

Example 24.42. *Illustrate associative law of (i) addition and (ii) multiplication as applied to Boolean algebra.*

Solution.

(*i*) The associative law of addition states that in the ORing of several variables, the

result is the same regardless of the grouping of the variables. This law for the addition of the three variables A, B and C can be written as :

$$A + (B + C) = (A + B) + C$$

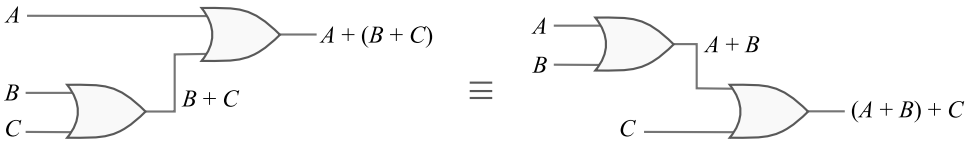


Fig. 24.62

Fig. 24.62. illustrates this law as applied to OR gates.

(ii) The associative law of multiplication states that it makes no difference in what order the variables are grouped when ANDing several variables. This law for the multiplication of the three variables A, B and C can be written as :

$$A\ (BC) = (AB)C$$

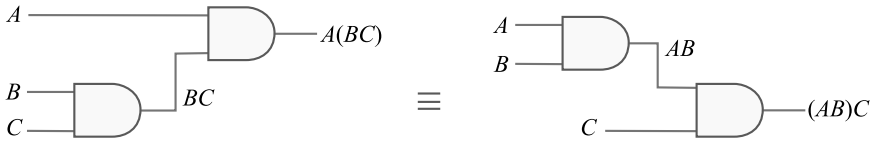


Fig. 24.63

Fig. 24.63 illustrates this law as applied to AND gates.

Example 24.43. *Illustrate distributive law as applied to Boolean algebra.*

Solution. This law states that ORing several variables and ANDing the result with a single variable is equivalent to ANDing the single variable with each of the several variables and ORing the products. The distributive law can be written for three variables (A, B and C) as under :

$$A\ (B + C) = AB + BC$$

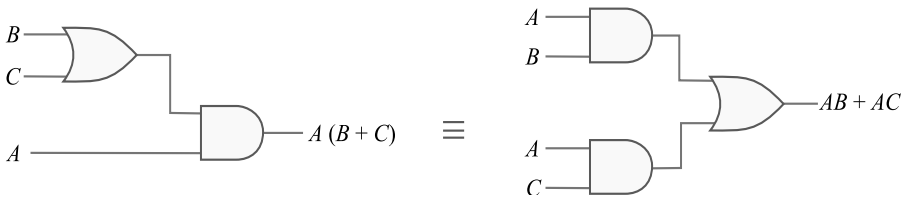


Fig. 24.64

Fig. 24.64 illustrates the distributive law in terms of gate implementation.

Note. The commutative law, the associative law and the distributive law of Boolean algebra are applicable regardless of the number of variables involved.

24.36 Truth Table from Logic Circuit

The output of a logic circuit for the given inputs can be determined directly from the circuit *without* using the Boolean expression. The procedure is to take a given set of inputs and work through each gate of the circuit to determine the final output. Let us illustrate the procedure with an example. Consider the logic circuit shown in Fig. 24.65. It consists of two AND gates and one OR gate. Each of the three gates has two input variables. Each of the input

variables can be either a high (1) or a low (0). Since there are four input variables, there are 16 possible combinations of the input variables ($2^4 = 16$). We shall develop the truth table for the circuit without using the Boolean expression for the circuit.

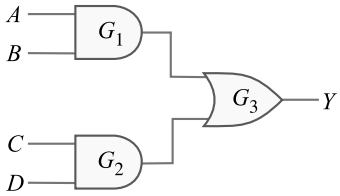


Fig. 24.65

First, suppose each input is low (*i.e.* $A = 0$, $B = 0$, $C = 0$ and $D = 0$). Under this condition, we shall examine the output of each gate in the circuit to arrive at the final output Y. If the inputs to gate G_1 are low (0), the output of G_1 is low (0). Also the output of G_2 is low (0) because both inputs are low (0). The two low inputs to G_3 also make its output low (0). Therefore, the output of the logic circuit is low when all the inputs are low as shown in the truth table. Similarly, we can find the output of the circuit for the remaining 15 input combinations.

Truth Table

Y = AB + CD

Inputs				G_1 *Output*	G_2 *Output*	G_3 *Output*
A	*B*	*C*	*D*	(*AB*)	(*CD*)	*Y*
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	0	1	1
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	0	1	1
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	1
1	1	0	1	1	0	1
1	1	1	0	1	0	1
1	1	1	1	1	1	1

Note. We can also construct the truth table by developing the Boolean expression from the logic circuit. The logic circuit shown above (See Fig. 24.65) is redrawn as shown in Fig. 24.66. The Boolean expression for this logic circuit is

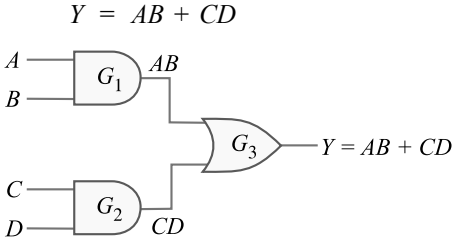


Fig. 24.66

Now simply put in the values for each combination of inputs and use the Boolean rules and laws to determine the final output *Y*. For example, when $A = 1$, $B = 1$, $C = 1$ and $D = 0$, the final output is

$$\begin{aligned} Y &= AB + CD \\ &= 1.1 + 1.0 \\ &= 1 + 0 \\ &= 1 \end{aligned}$$

Similary, we can find *Y* for the remaining 15 input combinations to get the truth table shown above.

24.37 Developing Logic Circuit from its Boolean Expression

We have seen how we can derive the Boolean expression for a given logic circuit. We now do the opposite *i.e.* create the logic circuit from its Boolean expression. Consider the following Boolean expression :

$$X = AB + CD$$

The function is composed of two terms, *AB* and *CD,* and it contains four variables. The first term is formed by ANDing *A* and *B* and the second term is formed by ANDing *C* and *D*. These two terms are then ORed to form the function *X*. The logic gates required to implement $X = AB + CD$ are as follows :

(***i***) Two two-input AND gates to form *AB* and *CD*.

(***ii***) One two-input OR gate to form the final function *X* (= *AB* + *CD*).

The logic circuit that performs the function $X = AB + CD$ is shown in Fig. 24.67.

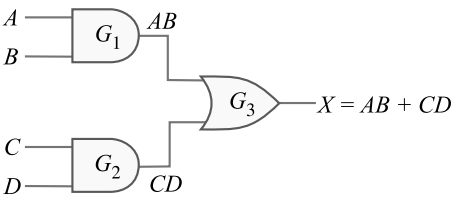


Fig. 24.67

Example 24.44. *Draw the logic circuit that implements the expression* $X = AB + CDE$.

Solution.

$$X = AB + CDE$$

The function is composed of two terms, *AB* and *CDE,* and it contains a total of five variables. The first term is formed by ANDing *A* and *B* and the second term is formed by ANDing *C*, *D* and *E*. These two terms are then ORed to form the function *X*. The logic gates required to implement $X = AB + CDE$ are as follows :

(*i*) One two-input AND gate to form *AB*.

(*ii*) One three-input AND gate to form *CDE*.

(*iii*) One two-input OR gate to form the final function *X* (= *AB* + *CDE*).

The logic circuit that performs the function $X = AB + CDE$ is shown in Fig. 24.68.

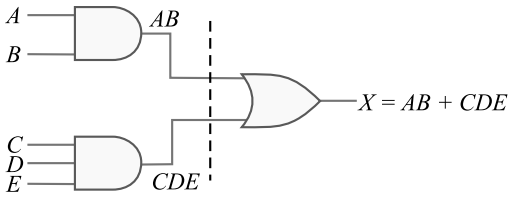


Fig. 24.68

Example 24.45. *Draw the logic circuit that implements the expression* $X = AB + \overline{B}C$.

Solution.

$$X = AB + \overline{B}C$$

The function is composed of two terms, *AB* and $\overline{B}C$, and it contains three variables. The first term is formed by ANDing *A* and *B* and the second term is formed by ANDing $\overline{B}$ and *C*. These two terms are then ORed to form the final function. The logic gates required to implement $X = AB + \overline{B}C$ are as follows :

(*i*) One inverter to form $\overline{B}$.

(*ii*) Two two-input AND gates to form *AB* and $\overline{B}C$.

(*iii*) One two-input OR gate to form the final function *X* (= $AB + \overline{B}C$).

The logic circuit that performs the function $X = AB + \overline{B}C$ is shown in Fig. 24.69.

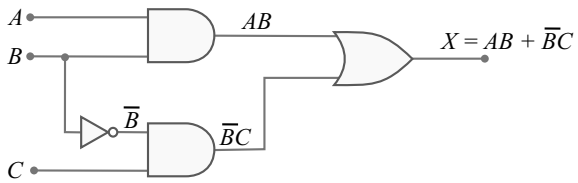


Fig. 24.69

Example 24.46. *Draw the logic circuit that implements the expression* $X = AB\,(C\overline{D} + EF)$.

Solution.

$$X = AB\,(C\overline{D} + EF)$$

A breakdown of this equation shows that the term *AB* and the term $C\overline{D} + EF$ are ANDed. The term *AB* is formed by ANDing the variables *A* and *B*. The term $C\overline{D} + EF$ is formed first by ANDing *C* and $\overline{D}$, ANDing *E* and *F* and then ORing these two terms. The

logic gates required to implement $X = AB\ (C\overline{D} + EF)$ are as follows :

(*i*) One inverter to form $\overline{D}$.

(*ii*) Two two-input AND gates to form $C\overline{D}$ and EF.

(*iii*) One two-input OR gate to form $C\overline{D} + EF$.

(*iv*) One two-input AND gate to form AB.

(*v*) One two-input AND gate to form X [$= AB\ (C\overline{D} + EF)$].

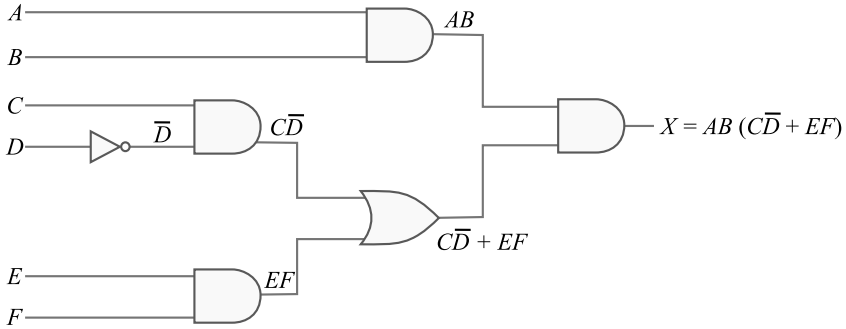


Fig. 24.70

The logic circuit that forms the function $X = AB\ (C\overline{D} + EF)$ is shown in Fig. 24.70.

24.38 Sum-of-Products Form

The sum-of-products form of a Boolean expression consists of two or more AND terms (*i.e.* products) that are ORed together. For example, $AB + CD$ is a sum-of-products expression. Here AND terms AB and CD are ORed (added). Other examples of this form are :

(*i*) $ABC + \overline{A}B\overline{C}$

(*ii*) $A\overline{B}C + DEF + A\ \overline{EF}$

(*iii*) $\overline{A} + BC\overline{D} + EFG$

The sum-of-products is a very useful form of a Boolean expression due to the straightforward manner in which it can be implemented in logic gates. For example, Fig. 24.71 shows the logic circuit that results in a sum-of-products form. It has simply two steps : ANDing and then ORing. Therefore, this form is always only a *two-level* gate network *i.e.,* the maximum number of gates through which a signal must pass in going form input to output is two (excluding inversions, but these can also be worked in).

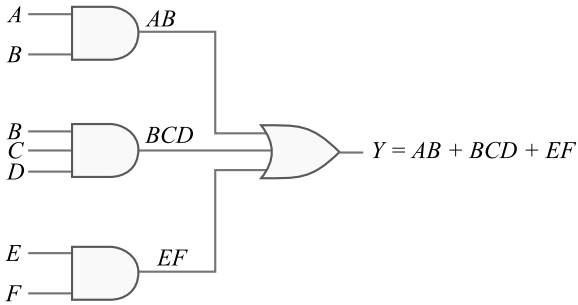


Fig. 24.71

24.39 Simplification of Boolean Expressions

The form of Boolean expression determines how many and which types of logic gates are needed as well as how they are connected together. The more complicated the Boolean expression, the more complex the logic circuit will be. It is, therefore, desirable to simplify an expression as much as possible to get the simplest logic circuit. Note that the new expression can be used to implement a logic circuit that is equivalent to the original logic circuit but contains fewer gates and connections. While simplifying a Boolean expression, the following two steps may be very helpful :

(*i*) Put the original expression into the sum-of-products form by the repeated use of rules, theorems and techniques of Boolean algebra.

(*ii*) Once it is in this form, the product terms are checked for common factors and factoring is performed wherever possible.

Illustration. Fig. 24.72 shows the logic circuit. The Boolean expression for this circuit is

$$X = AC\overline{D} + \overline{A}B\,(CD + BC)$$

We require *five AND gates*, *two OR gates* and *two inverters* to implement this expression. In all, nine gates are needed. We shall now use laws, rules and techniques of Boolean algebra to get the simplest expression for the given function.

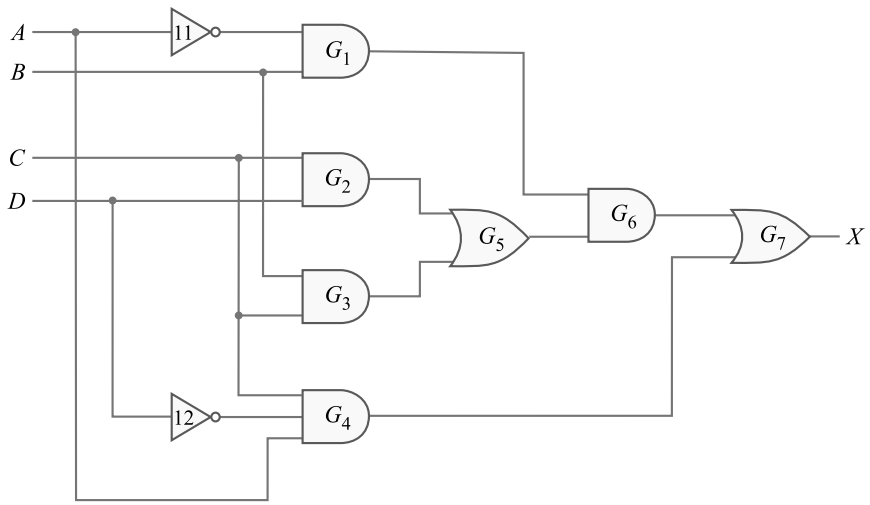


Fig. 24.72

Step 1. Apply distributive law to the second term by multiplying the term $CD + BC$ by $\overline{A}B$. The result is :

$$X = AC\overline{D} + \overline{A}BCD + \overline{A}BBC$$

Step 2. Applying the rule $BB = B$ to the third term, we have,

$$X = AC\overline{D} + \overline{A}BCD + \overline{A}BC$$

Step 3. Note that C is common to every term so that it can be factored out using distributive law.

$$\therefore \qquad X = C\,(A\overline{D} + \overline{A}BD + \overline{A}B)$$

Step 4. We see that the term $\overline{A}B$ appears in the last two terms within the bracket and can be factored out of those two terms.

$$\therefore \qquad X = C\,[A\overline{D} + \overline{A}B\,(D + 1)]$$

Since $D + 1 = 1$, $X = C\,(A\overline{D} + \overline{A}B)$

It appears that this equation cannot be simplified any further, but it can be written in a slightly different way by applying the distributive law (this results in the sum-of-products form) :

$$X = AC\overline{D} + \overline{A}BC$$

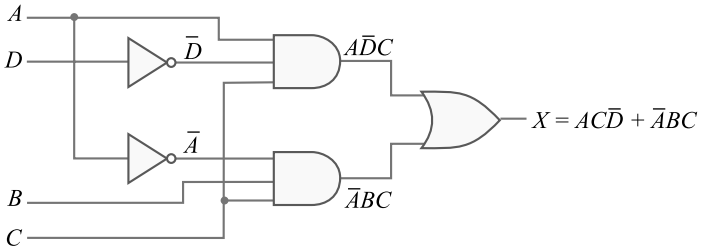


Fig. 24.73

Implementing this equation (*i.e.* $X = AC\overline{D} + \overline{A}BC$) into the logic circuit, it only requires *two three-input AND gates*, *two inverters* and *one two-input OR gate* as shown in Fig. 24.73. Note that this minimised circuit is equivalent to the original circuit of Fig. 24.72 but only requires five gates (instead of nine).

Example 24.47. *Simplify the expression :*

$$X = \overline{A}\overline{B}C + A\overline{B}C + AB\overline{C} + ABC$$

Solution.

$$X = \overline{A}\overline{B}C + A\overline{B}C + AB\overline{C} + ABC$$

Step 1. Note that the first two terms have $\overline{B}C$ as common factors while the last two terms have AB as common factors.

$$\therefore \qquad X = \overline{B}C\,(A + \overline{A}) + AB\,(C + \overline{C})$$

Step 2. $A + \overline{A} = 1$ and $C + \overline{C} = 1$ so that :

$$X = \overline{B}C \cdot 1 + AB \cdot 1$$

Step 3. Since $\overline{B}C \cdot 1 = \overline{B}C$ and $AB \cdot 1 = AB$ so that :

$$\boldsymbol{X = AB + \overline{B}C}$$

Note that not only is the Boolean expression simplified, but so is the resultant logic circuit.

Example 24.48. *Simplify the expression :*

$$X = AB + A\,(B + C) + B\,(B + C)$$

Solution.

$$X = AB + A\,(B + C) + B\,(B + C)$$

Step 1. Applying distributive law to the second and third terms, we have,

$$X = AB + AB + AC + BB + BC$$

Step 2. Now $BB = B$ and $AB + AB = AB$ so that :

$$X = AB + AC + B + BC$$

Step 3. $B + BC = B(1 + C) = B \cdot 1 = B$

$\therefore$ $$X = AB + AC + B$$

Step 4. Factoring B out, we have,

$$X = B(A + 1) + AC$$

Step 5. $A + 1 = 1$ so that $B(A + 1) = B \cdot 1 = B$.

$\therefore$ $$\boldsymbol{X = B + AC}$$

The original expression is simplified as far as it can go. Once you get acquainted with Boolean simplification techniques, you can combine many individual steps.

Example 24.49. *Simplify the circuit of Fig. 24.74 (i).*



(*i*) (*ii*)

Fig. 24.74

Solution. The Boolean expression for the circuit shown in Fig. 24.74 (*i*) is

$$X = (\overline{A} + B)(A + \overline{B})$$

Step 1. Multiplying out to get the sum-of-products form, we have,

$$X = \overline{A}A + \overline{A}\overline{B} + BA + B\overline{B}$$

Step 2. Now $\overline{A}A = 0$ and $B\overline{B} = 0$ so that :

$$X = \overline{A}\overline{B} + AB$$

This expression is implemented in Fig. 24.74 (*ii*). If you compare the circuit with the original circuit, you see that both circuits contain the same number of gates and connections. **Therefore, the simplification process has not produced a simpler circuit but it has produced an alternative circuit.**

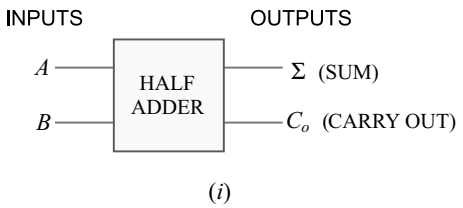
24.40 Electronic Adders

A logic circuit that performs the function of binary addition is called **electronic adder** *or* **adder.** The adder circuit consists of properly connected logic gates. There are usually two forms of the adder in common applications *viz.* **(*i*)** half adder **(*ii*)** full adder.

(*i*) Half Adder (HA). A logic circuit that can add two binary bits is called a half adder (*HA*). Fig. 24.75 (*i*) shows the block symbol for the half adder. The adder circuit would need two inputs and two outputs. The two inputs are for the two digits to be added,

* This is the alternate symbol for the inverter.

either 0 or 1. One output terminal is for the sum of the two inputs and the other output is for the carry, if necessary. Fig. 24.75 (*ii*) shows the addition table of the adder and can be thought of as a truth table. The numbers being added are on the input side of the table. The truth table has two output columns, one column for the sum and one column for the carry. The sum column is labelled with summation symbol Σ. The carry column is labelled with C_o. The C_o stands for *carry output* or *carry out*. Thus the half adder circuit has two inputs (A, B) and two outputs (Σ, C_o).



(*i*)

TRUTH TABLE

INPUTS		OUTPUTS	
B	*A*	Σ	C_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1
BINARY DIGITS TO BE ADDED		SUM	CARRY OUT
		XOR	AND

(*ii*)

Fig. 24.75

The half adder would behave according to the truth table shown in Fig. 24.75 (*ii*). Take a careful look at the truth table. The output columns (sum and carry) can be produced by using two gates as under :

(*a*) The sum column is the output of XOR gate. Remember that XOR gate has HIGH output when either input is HIGH but not when both inputs are the same.

(*b*) The carry column is the output of the AND gate. Both inputs must be HIGH for there to be a HIGH in the output.

Thus we can produce half adder using a two-input AND gate and a two-input XOR gate as shown in Fig. 24.76. The half adder circuit adds only the LSB column (1s column) in a binary addition problem. The reason is that it has no input for a carry-in.

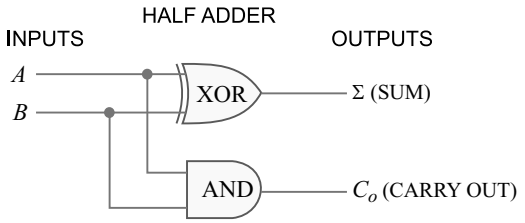
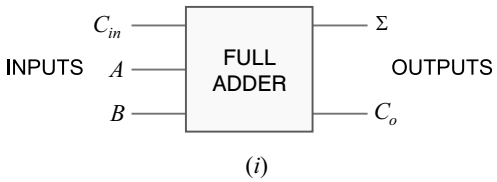


Fig. 24.76

(*ii*) Full Adder (FA). A full adder adds two binary bits plus a carry input (C_{in}) to produce the sum (Σ) and carry-out (C_o) outputs. Fig. 24.77 (*i*) shows the block diagram of a full adder. It is formed by using two half adder circuits and an OR gate as shown in Fig. 24.77 (*iii*). Note the carry-in input which requires the extra half adder. The output of the OR gate forms the carry-out (C_o) output.

The full adder has three inputs : C_{in} , A and B. These three inputs must be added to get the Σ and C_o outputs. Fig. 24.77 (*ii*) shows the truth table for a full adder. It shows all the possible combinations of A, B and C_{in}. The full adder is used for binary addition in all places except the 1s place which means 2s, 4s, 8s *etc*.

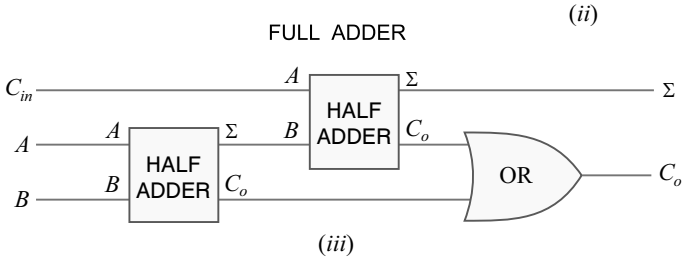


(*i*)

Truth Table

INPUTS			OUTPUTS	
C_{in}	B	A	Σ	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1
Carry + B + A			Sum	Carry out

(*ii*)



(*iii*)

Fig. 24.77

24.41 Flip-Flop (FF)

The most important memory or storage element is the *flip-flop* (abbreviated as *FF*) which is made up of an assembly of logic gates. *A* **flip-flop** *is a bistable circuit that can exist in either of two stable states (1 and 0) indefinitely and can be made to change its state by means of some external signal.* Because the flip-flop's output remains at either a 1 or a 0 depending on the last input, the flip-flop can be said to "remember" *i.e.,* it can act as a memory or storage element. The following points may be noted about flip-flops :

(***i***) Another name for the flip-flop is *bistable multivibrator*.

(***ii***) A flip-flop is a digital circuit (*i.e.,* it is made up of logic gates) with 2 outputs.

(***iii***) The two output signals of *FF* are opposite of each other. If one output is 1, the other output is 0 and *vice-versa*.

(***iv***) A flip-flop has normally two or more inputs.

(***v***) A flip-flop can be triggered to either state (1 or 0) and remains there after the triggering signal is removed. This property of *FF* accounts for the storage capability of *FF*.

24.42 Flip-Flop Terminology

There are several unique terms used to describe flip-flop operation. Fig. 24.78 shows the general type of symbol used for a flip-flop.

(***i***) A flip-flop has normally two or more inputs and two outputs. The two outputs, labelled Q and $\overline{Q}$, are opposite of each other. If Q = 1 (HIGH), then $\overline{Q}$ = 0 (LOW). If Q = 0, then $\overline{Q}$ = 1.

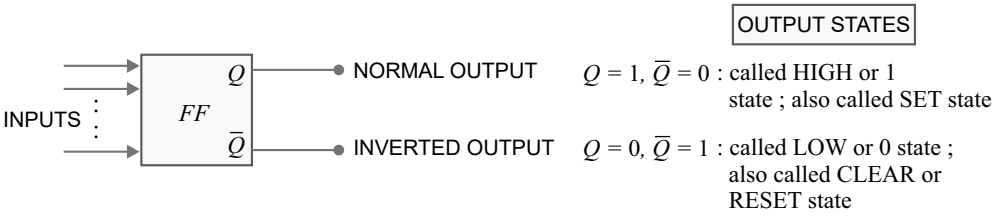


Fig. 24.78

(***ii***) The Q output is called the *normal FF output* and $\overline{Q}$ is the *inverted FF output*. Whenever we refer to the state of a flip-flop, we are referring to the state of its normal (Q) output ; it is understood that its inverted output ($\overline{Q}$) is in the opposite state. Thus when we say that *FF* is in HIGH state (1), it means $Q = 1$. If we say that *FF* is in LOW state (0), it means $Q = 0$. Of course, $\overline{Q}$ state will always be the opposite of Q.

(***iii***) A flip-flop is said to be *set* when $Q = 1$ and $\overline{Q} = 0$. Conversely, the flip-flop is *reset* when $Q = 0$ and $\overline{Q} = 1$. Thus the process of causing the flip-flop to go to the $Q = 1$, $\overline{Q} = 0$ state is called *setting* the flip-flop. On the other hand, causing the flip-flop to go to the $Q = 0$, $\overline{Q} = 1$ state is called *resetting* (this action is also referred to as "*clearing*") the flip-flop.

The circuit is called flip-flop because external triggering signals at the input of *FF* cause the *FF* to switch back and forth ("flip-flop") between its two possible stable output states.

24.43 SR Flip-Flop

The *SR* flip-flop is built using two cross-coupled NAND gates or NOR gates. The NAND gate version is called NAND gate *SR* flip-flop. Since the NAND and NOR *SR* flip-flops perform the same basic function, we shall limit our discussion of *SR* flip-flop to the NAND-type. Fig. 24.79 shows the circuit diagram of *SR* flip-flop using two NAND gates. The two NAND gates 1 and 2 are cross-coupled so that the output of NAND-1 is connected to one of the inputs of NAND-2 and *vice-versa*. The two inputs are labelled *S*(*set*) and *R*(*reset*) and the two outputs are Q and $\overline{Q}$. The logic symbol for *SR* flip-flop is shown in Fig. 24.80. The exact behaviour of a flip-flop is defined by its truth table. The truth table for a NAND *SR* flip-flop is shown in Fig. 24.81.

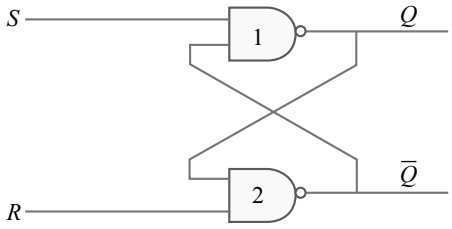


Fig. 24.79

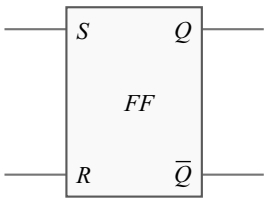


Fig. 24.80

(***i***) In Fig. 24.81, if both inputs to the flip-flop are LOW (*i.e.,* $S = 0$ and $R = 0$), both outputs will be HIGH (1). This is identified in the truth table as being "non allowed". This

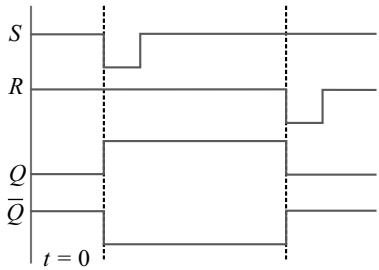
mode is not used because it drives both outputs HIGH (1) at the same time. This is not the defined operation of a flip-flop since a flip-flop must operate with complementary outputs.

(*ii*) When the set input is LOW and reset input is HIGH (*i.e.* $S = 0$, $R = 1$), the normal output Q of *FF* is HIGH (1) and the inverted output $\overline{Q}$ is LOW (0). Whenever the normal output Q of any flip-flop in HIGH (1), it is said to be **set**.

Truth table for NAND SR flip-flop

S	*R*	*Q*	$\overline{Q}$	State
0	0	1	1	Not allowed
0	1	1	0	Set
1	0	0	1	Reset
1	1	No change		Previous

Fig. 24.81



Timing diagram

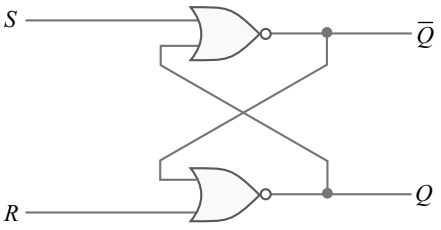
Fig. 24.82

(*iii*) When the set input is HIGH and reset input is LOW (*i.e.,* $S = 1$, $R = 0$), the normal output Q of *FF* is LOW (0) and output $\overline{Q}$ is HIGH (1). Whenever the normal output Q of any flip-flop is LOW (0), the flip-flop is said to be **reset**.

(*iv*) If both inputs to the *FF* are HIGH (*i.e.,* $S = 1$, $R = 1$), the present output is determined by the previous output. This is the idle or at rest condition and leaves Q and $\overline{Q}$ in their previous complementary states. This is called *disabled state* because there is *no change*. The outputs stay as they were, with Q either set or reset. Because of the disabled state, the flip-flop "remembers" the preceding state by remaining at that state until it is switched. This operation is possible because a flip-flop is a bistable circuit.

Fig. 24.82 shows the *timing diagram* or *waveform diagram* for NAND *SR* flip-flop. This diagram shows the voltage level and timing between inputs and outputs. The horizontal distance is *time* and vertical distance *voltage*.

Note. Fig. 24.83 shows the circuit of *SR* flip-flop using NOR gates. This circuit is implemented by cross-connecting two NOR gates in a manner that is very similar to the NAND gate version described above. The truth table for NOR gate *SR* flip-flop is shown in Fig. 24.84.



Logic symbol

Fig. 24.83

Truth table for NOR SR flip-flop

S	*R*	*Q*	$\overline{Q}$	State
0	0	No change		Previous
0	1	0	1	Reset
1	0	1	0	Set
1	1	1	1	Not allowed

Fig. 24.84

The main difference between the two (NAND type and NOR type) is that as the activity changes at the S and R inputs logic are 1's instead of logic 0's. Therefore, for the NOR

gate configuration, the quiescent state is $S = 0$, $R = 0$ such that no change will occur and the *not allowed* state is $S = 1$, $R = 1$. Since this state is not allowed, the output for this condition is often referred to as undefined. This state becomes important when considering logic minimisation techniques. Fig. 24.85 shows the timing diagram of NOR *SR* flip-flop.

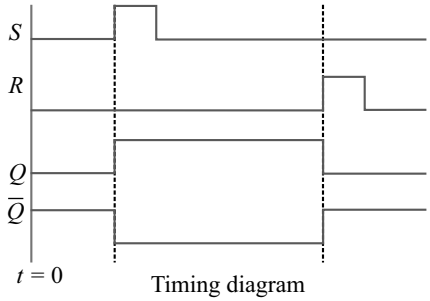
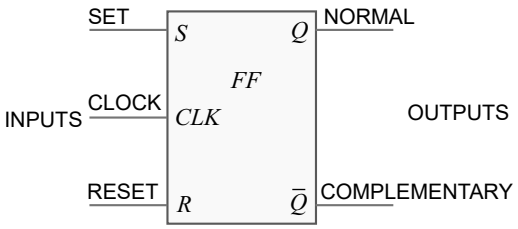


Fig. 24.85

24.44 Clocked SR Flip-Flop

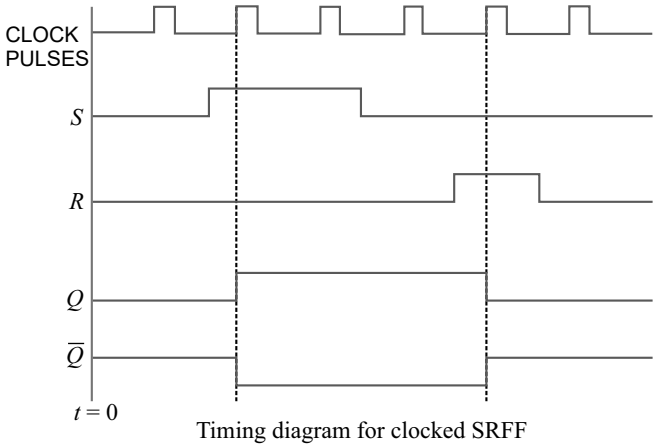
Basically, a clocked *SR* flip-flop operates in the same manner as the NAND and NOR gate versions with the exception that the clocked pulse (square wave) controls the times at which state changes can happen. The logic symbol for a clocked *SR* flip-flop is shown in Fig. 24.86. Note that it looks almost like *SR* flip-flop except that it has one extra input labelled *CLK* (for clock). With the simple *SR* flip-flop, the outputs change at the same time as the *R* and *S* inputs change. However, with a clocked flip-flop, the outputs do not change until a clock pulse is received on the *CLK* input. These clock pulses occur at some fixed interval (for example, every 10 microseconds) and all logic state changes are synchronised to occur at the time when the clock pulse occurs.



Logic symbol for a clocked R-S flip-flop

Fig. 24.86

Fig. 24.87 shows the timing diagram for a clocked *SR* flip-flop. It is clear from the timing diagram that Q and $\overline{Q}$ outputs do not respond directly to the *S* and *R* inputs ; rather they wait for the next clock pulse before changing states. For this flip-flop, if the output is in Q state when clock pulse n occurs, it will remain in that state for clock pulse $n + 1$ provided that inputs *S* and *R* are both zero. If either *S* or *R* goes to the logic 1 state, then the flip-flop will assume the appropriate state upon the occurrence of the next clock pulse. Finally, as with other *SR* flip-flops, there is a *not allowed* condition, namely $S = 1$, $R = 1$ for which the output is undefined. This condition is represented by a question mark in the truth table in Fig. 24.88.



Timing diagram for clocked SRFF

Fig. 24.87

S	R	Q^{n+1}
0	0	Q^n
0	1	0
1	0	1
1	1	?

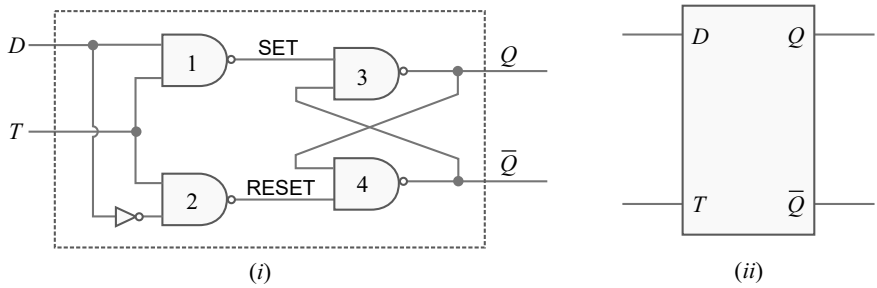
Truth table

Fig. 24.88

Note that the output of the clocked *SR* flip-flop *changes only on a clock pulse*. We say that this flip-flop operates *synchronously i.e.,* it operates *in step with* the clock. Synchronous operation is very important in calculators and computers where each step must happen in a very exact order.

24.45 D Flip-Flop

The *D* flip-flop is a slight modification of the *SR* flip-flop. If we add two NAND gates and an inverter to a NAND *SR* flip-flop as shown in Fig. 24.89 (*i*), the resulting circuit is called *D* flip-flop. The logic symbol for *D* flip-flop is shown in Fig. 24.89 (*ii*). The inputs for this type of flip-flop are labelled *D* and *T*. The outputs are labelled *Q* and $\overline{Q}$. The *T* input represents the input for a stable clock pulse. The *D* input represents the data input. As we shall see, when clock pulse is applied, the *Q* output of the flip-flop assumes the same logic level as the *D* input. If *D* = 1, then *Q* = 1 and if *D* = 0, then *Q* = 0.



(*i*) (*ii*)

Fig. 24.89

When the clock input is low, the *D* input will have no effect on the output, since the low at the *T* input causes the output of both NAND 1 and NAND 2 to go high. When the clock input is high, the *D* input will produce a low at either the set or reset of gates 3 and 4, depending on whether *D* input is high or low. When clock pulse is applied, the flip-flop follows the state of whatever data is present at the *D* input. In other words, when *T* = 1, *Q* follows the *D* input. If *D* = 1, the *Q* output of the flip-flop goes to the 1 state but if *D* = 0, the *Q* output of the flip-flop goes to 0 state. This fact is shown in the truth table for the *D* flip-flop in Fig. 24.90.

T	*D*	*Q*	$\overline{Q}$	State
0	0	No change		Previous
0	1	No change		Previous
1	0	0	1	Reset
1	1	1	0	Set

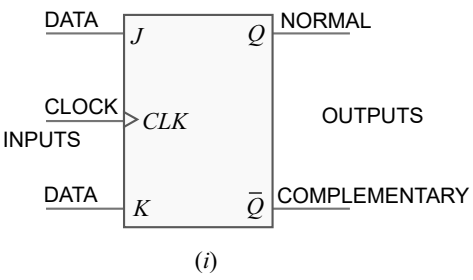
Fig. 24.90

The *D* flip-flop is often called a *delay flip-flop*. The word "delay" describes what happens to the data at input *D*. The data (a 0 or 1) at input *D* is *delayed one clock pulse* from getting to the output *Q*. In other words, the flip-flop can be considered to have stored the data from the time of the first clock pulse until such time as another clock pulse occurs.

D filp-flops are connected in series to form *shift-registers* or in parallel to form *data storage registers*. In the shift-register arrangement, delays of many clock periods can be obtained. When used as storage registers, the flip-flops can store data indefinitely or until required. Shift registers and storage registers are widely used in digital data handling circuits.

24.46 JK Flip-Flop

The *JK* flip-flop is probably the most widely used and is considered the universal flip-flop because it can be used in many ways. This type of device has a clock input as well as a *J* and *K* input. The *JK* flip-flop is a refinement of *SR* flip-flop in that *JK* flip-flop has four different operating states while *SR* flip-flop (NAND type of NOR type) has only three usable states. Inputs *J* and *K* behave like inputs *S* and *R* to set and reset the flip-flop respectively. The logic symbol for the *JK* flip-flop is shown in Fig. 24.91 (*i*). The inputs labelled *J* and *K* are the data inputs. The input labelled *CLK* is the clock input. Outputs *Q* and $\overline{Q}$ are the usual normal and complementary outputs. Note that *J* input is equivalent to the *S* (set) input of *SR* flip-flop and the *K* input is equivalent to the *R* (reset) input.



(*i*)

Truth table

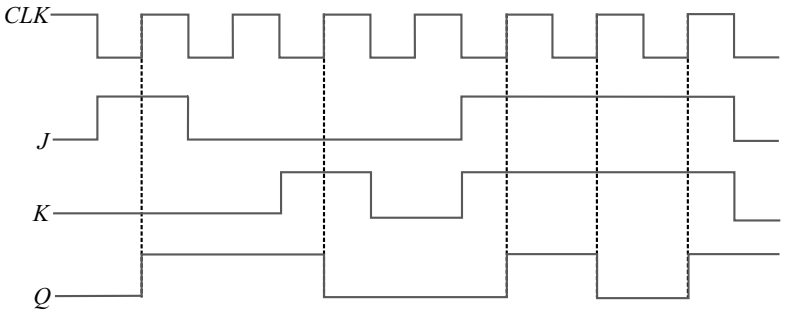
Mode of operation	*Inputs*			*Outputs*		
	CLK	*J*	*K*	*Q*	$\overline{Q}$	*Effect on output Q*
Hold	⊓	0	0	No change		No change – disable
Reset	⊓	0	1	0	1	Reset or cleared to 0
Set	⊓	1	0	1	0	Set to 1
Toggle	⊓	1	1	Toggle		Changes to opposite state

(*ii*)

Fig. 24.91

Fig. 24.91 (*ii*) shown the truth table for the *J-K* flip-flop. This table shows four useful modes of operation. When the *J* and *K* inputs are both 0, the flip-flop is in the *hold* (or *disabled*) *mode*. In the hold mode, the data inputs have no effect on the outputs. The outputs "hold" the last data present. Lines 2 and 3 of the truth table show the reset and set conditions for the *Q* output. Line 4 of the truth table illustrates the useful *toggle* position of

the *J–K* flip-flop. When both data inputs *J* and *K* are at 1, repeated *clock pulses cause the output to turn off-on-off-on-off-on and so on. This off-on action is like a toggle switch and is called *toggling*. Each clock pulse toggles the outputs to switch to their opposite states. *This condition is known as race around condition* Fig. 24.92 shows the timing diagram of *JK* flip-flop.

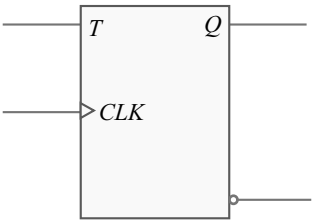


Timing diagram

Fig. 24.92

24.47 T Flip-Flop

Another type of flip-flop found in practice is the *T* (toggle) flip-flop. This flip-flop has the ability to change states (toggle) each time a clock pulse occurs. This flip-flop is obtained from the *JK* type when inputs *J* and *K* are connected to provide a single input designated by *T* as shown in Fig. 24.93 (*i*). The input *T* determines whether or not the toggling action will occur.



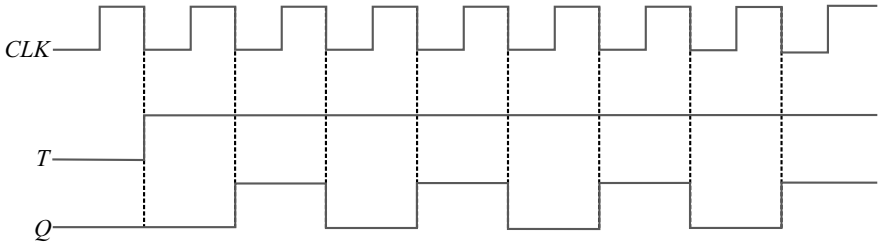
Logic symbol
(*i*)

T	*Q (t + 1)*	
0	*Q*(*t*)	No change
1	*Q*′(*t*)	Complement

Truth table
(*ii*)

Fig. 24.93

Fig. 24.93 (*ii*) shows the truth table for *T* flip-flop. When $T = 0$ ($J = K = 0$), the clock pulse does not change the state of the flip-flop. When $T = 1$ ($J = K = 1$), the flip-flop toggles with each clock pulse. Fig. 24.94 shows the timing diagram of *T* flip-flop.



Timing diagram

Fig. 24.94

* The clock pulses occur at some fixed interval, say after every 10 microsecond.

Short Answer Questions

Q. 1. *What do you mean by encoding and decoding ?*

Ans. Digital circuits understand only binary numbers *i.e.,* input to a digital circuit must be in binary form. *Encoding* means changing decimal numbers to a binary form. In *decoding*, the digital information is converted back into decimal numbers.

Q. 2. *Give two advantages of digital circuits over the analog circuits.*

Ans. The two advantages of digital circuits over analog circuits are :

(*i*) Information can be stored for short and indefinite period.

(*ii*) Digital circuits are very accurate.

Q. 3. *What are the limitations of digital circuits ?*

Ans. The following are the limitations of digital circuits as compared to analog circuits:

(*i*) Most of "real world" information dealing with time, speed, pressure and position measurements are analog in nature.

(*ii*) Analog processing is generally simpler and faster.

Q. 4. *What is a truth table ?*

Ans. The action of a logic gate is usually summarised in the form of a truth table. A truth table defines the output of a logic gate for all possible combinations of inputs.

Q. 5. *What is an inverter ?*

Ans. The NOT gate is also known as an inverter. It has only one input and one output. The output of an inverter is always opposite to its input *i.e.,* if input is 1 (HIGH), the output is 0 (LOW) and *vice-versa*.

Q. 6. *What is the importance of digital techniques ?*

Ans. A continuously varying signal (voltage or current) is called an *analog signal*. Due to many-valued output, the analog operation is less reliable. On the other hand, a signal that can have only two discrete values is called a *digital signal*. Since digital operation has only two states, it is far more reliable than many-valued analog operation.

Q. 7. *Give three examples of (i) analog devices (ii) digital devices.*

Ans. **(*i*)** Amplifiers, oscillators and radio receivers.

(*ii*) Calculators, electronic watches and computers.

Q. 8. *What is a logic gate ?*

Ans. A logic gate is a digital circuit which works according to some logical relationship between input and output voltages. It has one or more input signals but only one output signal.

Q. 9. *State the rule used in the operation of (i) OR gate (ii) AND gate (iii) NOT gate.*

Ans. **(*i*)** The output of an OR gate is high (1) if one or more inputs are high (1). However, the output is low (0) when all inputs are low (0).

(*ii*) The output of an AND gate is high (1) if all the inputs are high (1). However, the output is low (0) if any or all inputs are low (0).

(*iii*) The output of a NOT gate is high (1) if input is low (0) and *vice-versa*.

Q. 10. *Write the Boolean expression for (i) OR gate (ii) AND gate (iii) NOT gate.*

Ans. **(*i*)** $A + B = Y$ **(*ii*)** $A \,.\, B = Y$ **(*iii*)** $\overline{A} = Y$.

Q. 11. *Write the truth table for (i) OR gate (ii) AND gate and (iii) NOT gate.*

Ans. **(*i*)** Truth Table for **(*ii*)** Truth Table for **(*iii*)** Truth Table for

OR gate

Inputs		Output
A	B	$Y = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

AND gate

Inputs		Output
A	B	$Y = A \,.\, B$
0	0	0
0	1	0
1	0	0
1	1	1

NOT gate

Input	Output
A	Y
0	1
1	0

Fig. 24.95

The required truth tables for the logic gates are shown in Fig. 24.95.

Q. 12. *When is the output of (i) OR gate high (ii) NAND gate low ?*

Ans. **(*i*)** The output of OR gate is high (1) when any or all the inputs are high (1). **(*ii*)** The output of NAND gate is low (0) when both the inputs are high (1).

Q. 13. *How 2-input NAND gate can be converted into a NOT gate ?*

Ans. When two inputs of NAND gate are connected together so that it has one input, the NAND works as a NOT gate. Symbolically, it is shown in Fig. 24.96.

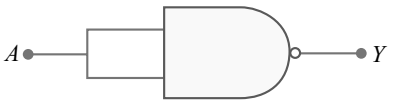


Fig. 24.96

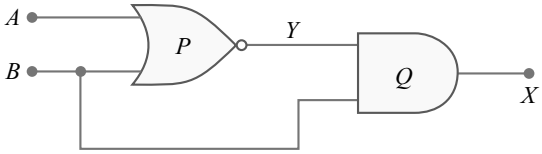
Q. 14. *Why is logic gate called so ?*

Ans. The term "logic" is usually used to refer to a decision-making process. A logic gate makes logical decisions regarding the existence of output depending upon the nature of the input. For this reason, it is called a logic gate.

Q. 15. *Why are the NAND (or NOR) gates called as digital building blocks ?*

Ans. The repeated use of NAND (or NOR) gates can produce all the three basic gates *i.e.,* OR, AND and NOT gates. Hence the NAND and NOR gates are called universal gates.

Q. 16. *Identify the logic gates marked P and Q in Fig. 24.97. Write down the output at X for the inputs (i) A = 0, B = 0 and (ii) A = 1, B = 1.*



A	B	Y	X
0	0	1	0
1	1	0	0

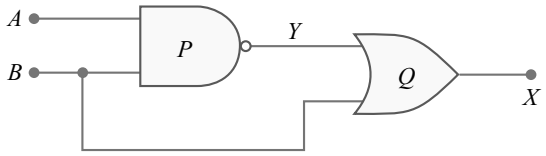
Fig. 24.97

Ans. The logic gate marked as P is NOR gate while the gate marked as Q is AND gate. It is clear that :

$$Y = \overline{A+B} \text{ and } X = B \cdot Y$$

Therefore, the output at X will be as shown in the truth table.

Q. 17. *Identify the logic gates marked P and Q in Fig. 24.98. Write down the output at X for the inputs (i) A = 0, B = 0 and (ii) A = 1, B = 1.*



A	B	Y	X
0	0	1	1
1	1	0	1

Fig. 24.98

Ans. The logic gate marked as P is NAND gate while the gate marked as Q is OR gate. It is clear that :

$$Y = \overline{A \cdot B} \text{ and } X = B + Y$$

Therefore, the output at X will be as shown in the truth table.

Q. 18. *How will you obtain T flip-flop from JK flip-flop ?*

Ans. The T flip-flop is obtained from JK flip-flop when inputs J and K are connected to provide a single input designated by T.

Multiple-Choice Questions

1. The binary number 10101 is equivalent to decimal number .....
(*i*) 19 (*ii*) 12
(*iii*) 27 (*iv*) 21

2. The universal gate is .....
(*i*) NAND gate (*ii*) OR gate
(*iii*) NOT gate
(*iv*) none of the above

3. The inverter is .....
(*i*) NOT gate (*ii*) OR gate
(*iii*) AND gate
(*iv*) none of the above

4. The inputs of a NAND gate are connected together. The resulting circuit is .....
(*i*) OR gate (*ii*) AND gate
(*iii*) NOT gate
(*iv*) none of the above

5. The NOR gate is OR gate followed by .....
(*i*) AND gate (*ii*) NAND gate
(*iii*) NOT gate
(*iv*) none of the above

6. The NAND gate is AND gate followed by .....
(*i*) NOT gate (*ii*) OR gate
(*iii*) AND gate
(*iv*) none of the above

7. Digital circuit can be made by the repeated use of .....
(*i*) OR gates (*ii*) NOT gates
(*iii*) NAND gates
(*iv*) none of the above

8. The only function of NOT gate is to .....
(*i*) stop a signal
(*ii*) invert input signal
(*iii*) act as a universal gate
(*iv*) none of the above

9. When an input signal 1 is applied to a NOT gate, the output is .....
(*i*) 0
(*ii*) 1
(*iii*) either 0 or 1
(*iv*) none of the above

10. In Boolean algebra, the bar sign (–) indicates .....
(*i*) OR operation
(*ii*) AND operation
(*iii*) NOT operation
(*iv*) none of the above

11. The given Boolean expression is :

$$Y = A\overline{B} + B\overline{A}$$

If $A = 1$ and $B = 1$, then Y = ...
(*i*) 1 (*ii*) 0
(*iii*) either 1 or 0
(*iv*) none of the above

12. In Boolean algebra, the plus sign (+) indicates .....
(*i*) AND operation
(*ii*) OR operation
(*iii*) NOT operation
(*iv*) none of the above

13. $\overline{(A + B)}$ = ...
(*i*) $\overline{A} + \overline{B}$ (*ii*) $\overline{A} - \overline{B}$
(*iii*) $\overline{A} \cdot \overline{B}$
(*iv*) none of the above

14. $\overline{(A \cdot B)}$ = ...
(*i*) $\overline{A} + \overline{B}$ (*ii*) $\overline{A} \cdot \overline{B}$
(*iii*) $\overline{A} - \overline{B}$
(*iv*) none of the above

15. $A + A \cdot B$ = ...
(*i*) B (*ii*) A
(*iii*) $\overline{A} + B$
(*iv*) none of the above

Answers to Multiple-Choice Questions

1. (*iv*)	**2.** (*i*)	**3.** (*i*)	**4.** (*iii*)	**5.** (*iii*)
6. (*i*)	**7.** (*iii*)	**8.** (*ii*)	**9.** (*i*)	**10.** (*iii*)
11. (*ii*)	**12.** (*ii*)	**13.** (*iii*)	**14.** (*i*)	**15.** (*ii*)

Chapter Review Topics

1. Write a short note on analog and digital signals.
2. What is a digital circuit ?
3. What is binary number system ?
4. How will you make decimal to binary conversion ?
5. How will you make binary to decimal conversion ?
6. What is a logic gate ?
7. What are the three basic logic gates ?
8. Describe OR function with a 2-input OR gate.
9. Explain AND function with a 2-input AND gate.
10. What is a NAND gate ?
11. What is a NOR gate ?
12. How will you obtain NOT gate from NAND gate ?
13. What is indicated by plus (+), dot (.) and bar (—) in a Boolean expression ?
14. State De Morgan's theorems.
15. What are encoders and decoders ?

Problems

1. Convert decimal number 23 into equivalent binary number. **[$(10111)_2$]**

2. Simplify the expression $Y = A\,C\,D + \overline{A}\,B\,C\,D$. **[$Y = A\overline{C} + \overline{B}D$]**

3. Simplify the expression $Y = \overline{(\overline{A} + C) \cdot (B + \overline{D})}$ to one having only single variables inverted. **[$Y = A\overline{C} + \overline{B}D$]**

4. Find the complement function of $Y = \overline{A}\,B\,\overline{C} + \overline{A}\,\overline{B}\,C$. **[$(A + \overline{B} + C)\,(A + B + \overline{C})$]**

5. Simplify the expression $Y = A \cdot B + A \cdot \overline{B}$. **[$Y = A$]**

6. Simplify the expression $Y = A \cdot B \cdot C + B \cdot C$. **[$Y = B \cdot C$]**

7. Simplify the following Boolean expression to a minimum number of literals :

$$Y = A\,(\overline{A} + B)$$ **[$Y = AB$]**

8. Simplify the following Boolean function to a minimum number of literals :

$$Y = \overline{A}\,\overline{B}\,C + \overline{A}\,B\,C + A\,\overline{B}$$ **[$Y = \overline{A}\,C + A\overline{B}$]**

9. Simplify the expression : $Y = (A + B)\,(\overline{A} + C)\,(B + C)$ **[$Y = (A + B)\,(\overline{A} + C)$]**

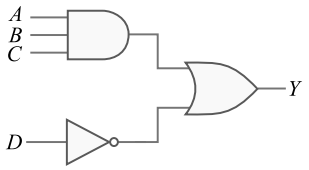
10. Find the complement of the function :

$$Y = A\,(\overline{B}\,\overline{C} + B\,C)\,H$$ **[$\overline{Y} = \overline{A} + (B + C)\,(\overline{B} + \overline{C})$]**

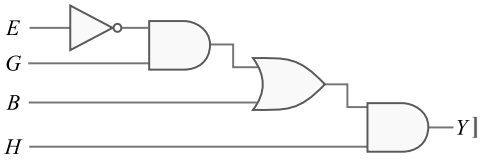
11. Draw the logic circuit for the following Boolean expressions :

(*i*) $Y = ABC + \overline{D}$ (*ii*) $Y = (\overline{E}G + B)\,H$

[**Ans.**



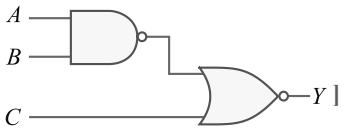
(*i*)



(*ii*)

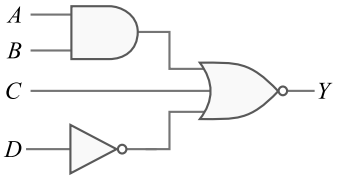
12. A logic circuit is given by the Boolean escperssion : $Y = \overline{\overline{AB} + C}$. Draw the logic circuit for this expression.

[**Ans.**

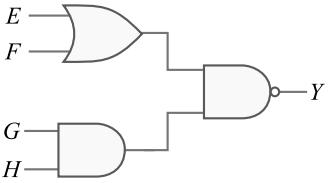


(*iii*)

13. Write the Boolean expressions for the logic circuits shown in Fig. 24.99.



(*i*)



(*ii*)

Fig. 24.99

[(*i*) $Y = \overline{AB + C + \overline{D}}$ (*ii*) $Y = \overline{(E + F)(GH)}$]

14. Write the Boolean expression for the logic circuit shown in Fig. 24.100.

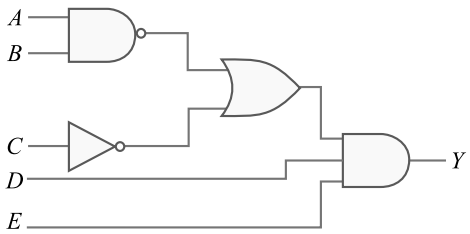


Fig. 24.100

[$Y = (\overline{AB} + \bar{C})\ DE$]

15. Write the BCD code for 829. **[100000101001_{BCD}]**

16. What is the decimal number for 10000111000 BCD ? **[$(438)_{10}$]**